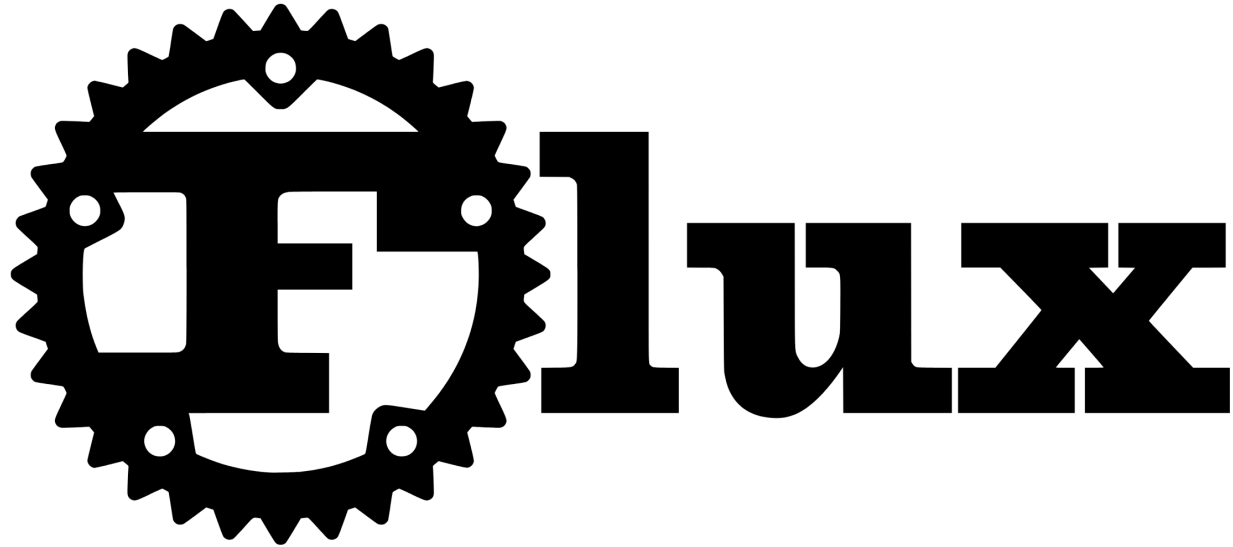
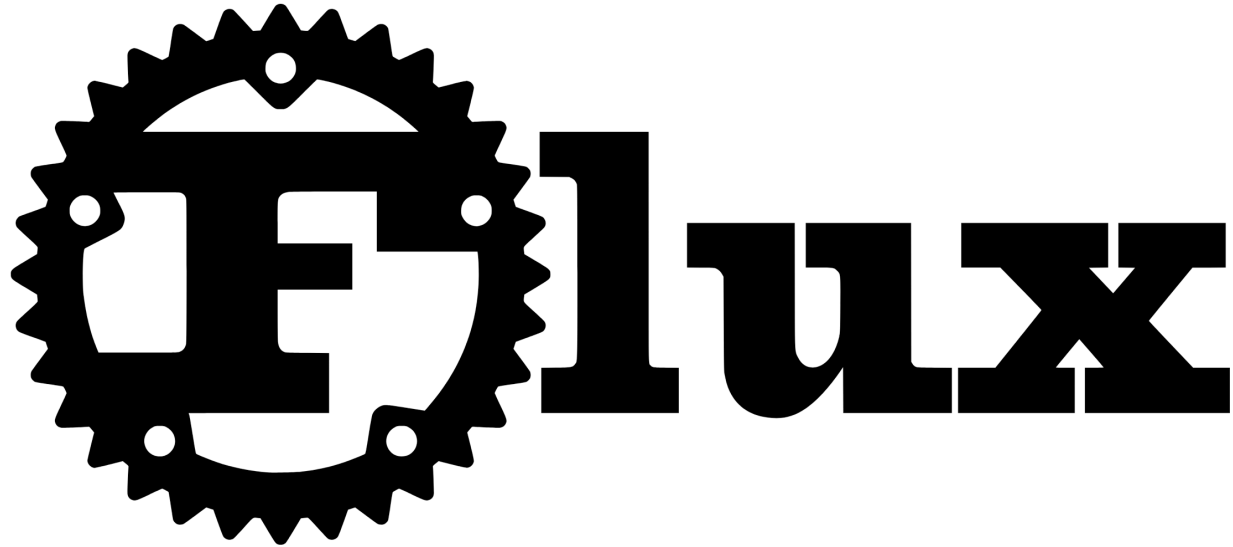


Refinement Types for Verified Systems



Ranjit Jhala, *UC San Diego*

Program Verification for the 21st Century



Ranjit Jhala, *UC San Diego*

Hoare Logic (1969)

An Axiomatic Basis for Computer Programming

C. A. R. HOARE

The Queen's University of Belfast, Northern Ireland*

In this paper an attempt is made to explore the logical foundations of computer programming by use of techniques which were first applied in the study of geometry and have later been extended to other branches of mathematics. This involves the elucidation of sets of axioms and rules of inference which can be used in proofs of the properties of computer programs. Examples are given of such axioms and rules, and a formal proof of a simple theorem is displayed. Finally, it is argued that important advantages, both theoretical and practical, may follow from a pursuance of these topics.

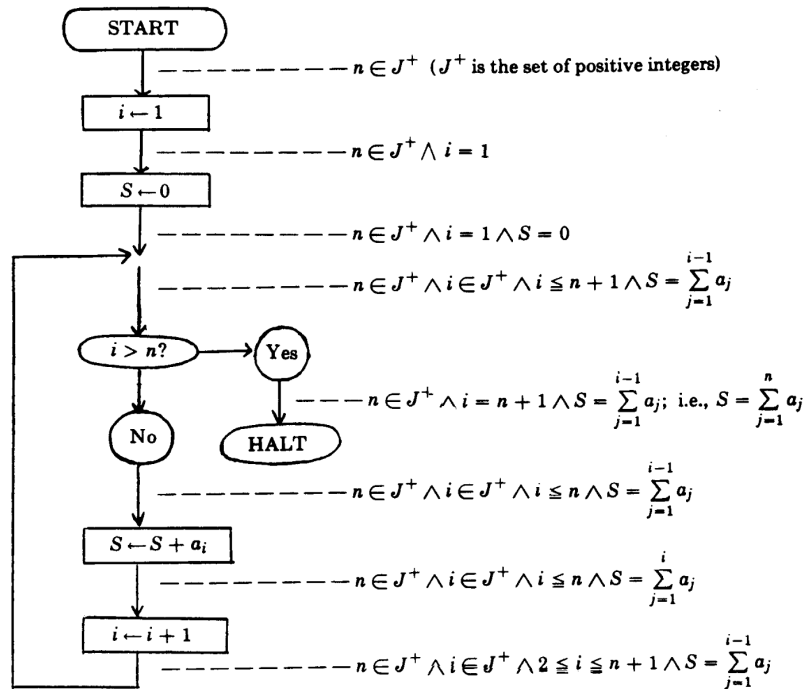
Hoare Logic (1969)

$$\{P\} \quad c \quad \{Q\}$$

Precondition Command Postcondition

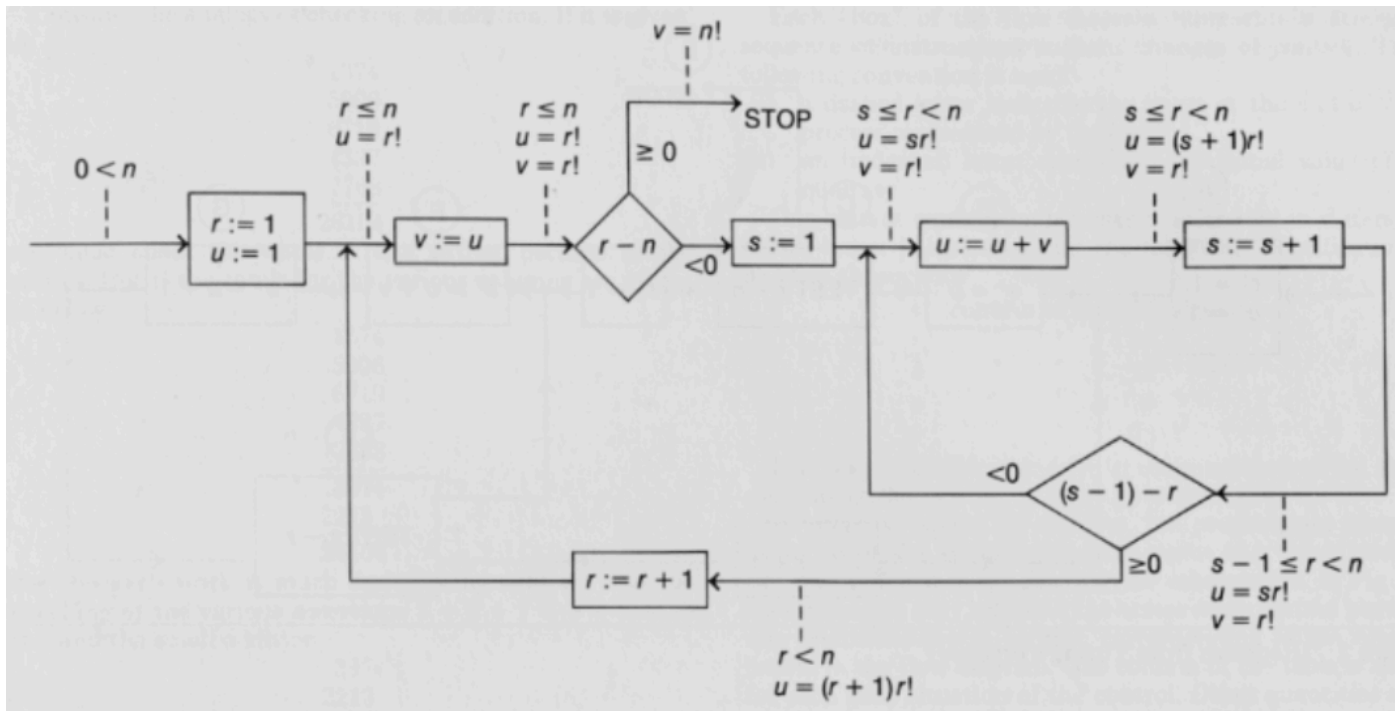
Floyd (1967)

“Assigning Meanings to Programs”



Turing (1949)

“Checking a Large Routine”



The Problem

Program Logic vs. 21st Century Code

Program Logic[★] *vs.* 21st Century Code

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Program Logic[★] *vs.* 21st Century Code

Iterations,

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Program Logic[★] *vs.* 21st Century Code

Iterations, Collections,

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Program Logic[★] *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Program Logic[★] vs. 21st Century Code

Iterations, Collections, λ -Abstractions, ...

[★] Illustrated using Verus, a Rust verifier similar to Dafny

Iteration using while

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
{
    let mut sum = 0.0;
    let mut i = 0;
    while i < x.len()
    {
        sum += x[i] * y[i];
        i += 1;
    }
    sum
}
```

Goal: Verify vector access *in-bounds*

Iteration using while

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
    invariant i <= x.len() && x.len() == y.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

Feasible: Need invariant on *explicit* iterator state!

Iteration using for

```
fn dot_product(x: &Vec<f64>, y: &Vec<f64>) → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  for i in 0..x.len()
    invariant ???
  {
    sum += x[i] * y[i];
  }
  sum
}
```

Problem: Need invariant on *implicit* iterator state!

Program Logic *vs.* 21st Century Code

Iterations, *Collections*, λ -Abstractions, ...

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>) → Vec<f64>

{
    let mut res = Vec::new();
    let mut i = 0;
    while i < ws.len()

    {
        let weighted_input = dot_product(&ws[i], ins);
        res.push(sigmoid(weighted_input));
        i += 1;
    }
    res
}
```

Goal: Verify *precondition* of dot_product

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>) → Vec<f64>
  requires forall |i: int| 0 <= i < ws.len() ==> ws[i].len() == ins.len()
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
    invariant
      i ≤ ws.len(),
      forall |j: int| 0 <= j < ws.len() ==> ws[j].len() == ins.len(),
    {
      let weighted_input = dot_product(&ws[i], ins);
      res.push(sigmoid(weighted_input));
      i += 1;
    }
  res
}
```

Problem: Quantified contracts (hard to write & verify)

Program Logic *vs.* 21st Century Code

Iterations, Collections, *λ -Abstractions*, ...

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
```

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize) → Vec<Vec<f64>> {
    init(out_size, |_| {
        init(in_size, |_| random())
    })
}
```

λ -Abstractions

```
// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize) → Vec<Vec<f64>> {
    init(out_size, |_| {
        init(in_size, |_| random())
    })
}

fn init<A>(n: usize, f: impl FnMut(usize) → A) → Vec<A> {
    let mut res = Vec::new();
    for i in 0 .. n {
        res.push(f(i));
    }
    res
}
```

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>

{
    let mut res = Vec::new();
    let mut i = 0;
    while i < n

        {
            res.push(f(i));
            i += 1;
        }
    res
}
```

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
    forall|i: int| 0 <= i < n ==> call_requires(f, i),
  ensures
    res.len() = n,
    forall|i: int| 0 <= i < n ==> call_ensures(f, i, res[i])
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < n
    invariant
      i ≤ n,
      res.len() = i,
      forall|i: int| 0 <= i < n ==> call_requires(f, i),
      forall|j: int| 0 <= j < i ==> call_ensures(f, j, res[j])
    {
      res.push(f(i));
      i += 1;
    }
  res
}
```

Problem: Hairy *higher-order* contracts!

Program Logic *vs.* 21st Century Code

Iterations, Collections, λ -Abstractions, ...

1. Problem

Program Logic *vs.* 21st century features

1. Problem

Program Logic vs. 21st century features

2. Solution

Refinements: *Weave* assertions into types

Refinements

Weave **Assertions** into **Types**

Weave *Assertions* into *Types*

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

1. *Refinements*

Index specifies *single value*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Index specifies *single value*

Index specifies *single value*

B [*v*]

Base Type *Refine* Index

Index specifies *single value*

i32[**5**]

The *singleton* **i32** that is equal to **5**

Index specifies *single value*

`bool[true]`

The *singleton* `bool` that is equal to `true`

Index specifies *single value*

```
fn tt() → bool[true] {  
    1 < 2  
}
```

Output type specifies *Postcondition*

A function that *always returns* **true**

Index specifies *single value*

```
fn ff() → bool[false] {  
    2 < 1  
}
```

Output type specifies *Postcondition*

A function that *always returns* false

Index specifies *single value*

```
fn twelve() → i32[12] {  
    4 + 8  
}
```

Output type specifies *Postcondition*

A function that *always returns* 12

Index specifies *single value*

```
fn assert(b:bool[true]){} }
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

```
fn assert(b:bool[true]){}  
  
...  
assert(1 < 2);  
assert(10 < 2); // flux error!
```

Input type specifies *Precondition*

A function that *requires* input be **true**

Index specifies *single value*

Constants are *boring*!

Index specifies *single value*

Constants are *boring*!

Parameters abstract over *input values*

Parameters abstract over *input values*

Parametric Refinement

`forall<n: int> fn (i32[n]) → bool[n > 0]`

Parameters abstract over *input values*

Parametric Refinement

`forall<n: int> fn (i32[n]) → bool[n > 0]`

Declared with @-syntax

`fn (i32[@n]) → bool[n > 0]`

Parameters abstract over *input values*

Parametric Refinement

`forall<n: int> fn (i32[n]) → bool[n > 0]`

Declared with @-syntax

`fn (i32[@n]) → bool[n > 0]`

Desugared from Rust

`fn (n:i32) → bool[n > 0]`

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
  n > 0  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn is_pos(n:i32) → bool[n>0] {  
    n > 0  
}  
  
...  
assert(is_pos(5));           // ok  
assert(is_pos(5 - 8));      // error
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
  n + 1  
}
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok
```

Parameters abstract over *input values*

Output's type *depends on input*

```
fn incr(n:i32) → i32[n+1] {  
    n + 1  
}  
  
...  
assert(incr(5 - 5) > 0); // ok  
assert(incr(5 - 6) > 0); // error
```

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Index specifies *single value*

B[*v*]

Index specifies *single value*

B [*v*]

But what if we *don't know* the exact value?

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Existential specifies *sets of values*

Constable & Smith 1987, Rushby et al. 1997

Existential specifies *sets of values*

$$\mathbf{B} \{ v : p(v) \}$$

Base Type Constraint

Existential specifies *sets of values*

$$\mathbf{B} \{ v : p(v) \}$$

Set of $\mathbf{B}$ values whose index v satisfies $p(v)$

Existential specifies *sets of values*

i32 { $v : 0 \leq v$ }

i32 values that are *non-negative*

Existential specifies *sets of values*

`usize`{`v`: `v < n`}

`usize` values *less than* `n`

Existential specifies *sets of values*

`abs` returns a non-negative `i32`

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}
```

Existential specifies *sets of values*

abs returns a non-negative **i32**

```
fn abs(n:i32) → i32{v:0 ≤ v} {  
  if n < 0 {  
    0 - n  
  } else {  
    n  
  }  
}  
  
...  
assert(abs(n) ≥ 0);  
assert(abs(n) >= n); // EX: How to fix?
```

Exercise: How can we *fix* the spec for *abs*?

Existential specifies *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T], i: usize) → &T
{
    &x[i] // EX: How to fix?
}
```

Exercise: What is a suitable type for the input *i*?

Existential specifies *sets of values*

get requires a valid index!

```
fn get<T>(x: &[T][@n], i: usize{v:v<n}) → &T
{
    &x[i]
}
```

Precondition: *i less than size of slice n*

1. *Refinements*

Index specifies *single value*

Parameters abstract over *input values*

Existential specifies unknown *sets of values*

Weave **Assertions** into **Types**

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

2. Ownership

2. *Ownership*

Update types at *owned locations*

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrowings*

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Update types at *owned locations*

Update types at *owned locations*

```
let mut x = 0;    // x : i32[0]  
assert(x == 0);
```

Exclusive ownership allows strong updates

Update types at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
```

Exclusive ownership allows strong updates

Update types at *owned locations*

```
let mut x = 0;      // x : i32[0]
assert(x == 0);
x += 10;            // x : i32[10]
assert(x == 10);
x += 10;            // x : i32[20]
assert(x == 20);
```

Exclusive ownership allows strong updates

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrow*s

Strong updates at *mutable borrow*s

Preserve refinements in *borrows*

Preserve refinements in *borrow*s

```
fn read(x: &i32{v: 0 < v}) {  
    let n = *x;  
    assert(0 < n);  
}
```

& reference = aliasing but *no mutation*

Value *read* through & reference satisfies refinements

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Strong updates at *mutable borrows*

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]  
    incr(&z);  
    assert(z == 2); // z: i32[2]  
}
```

Strong updates at *mutable borrows*

```
fn incr(x: &mut i32[@n]) ensures x: i32[n+1] {  
    *x += 1;  
}  
  
fn test() {  
    let mut z = 1; // z: i32[1]  
    incr(&z);  
    assert(z == 2); // z: i32[2]  
}
```

Sound as `&mut` is *unique* (no aliasing)

2. *Ownership*

Update types at *owned locations*

Preserve refinements in *borrows*

Strong updates at *mutable borrows*

Weave **Assertions** into **Types**

1. *Refinements* `i32`, `bool`, ...

2. *Ownership* `mut`, `&`, `&mut`, ...

3. *Example* revisited

3. Example revisited

Types make specification & verification *compositional*

struct: *Refined Vectors*

```
struct RVec<T> {  
    inner: Vec<T>;  
}
```

RVec is a *refined wrapper* around standard library Vec

struct: *Refined Vectors*

```
#[refined_by(len: int)]  
struct RVec<T> {  
    inner: Vec<T>;  
}
```

refined_by: *refinement value(s)* tracked for RVec<T>

Refined Vectors: Specification

Refined Vectors: *Specification*

```
impl RVec<T> {
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;  
  
    // insert element at the end of the vector  
    fn push(&mut self[@n], val: T) ensures self: RVec<T>[n+1];  
}
```

Refined Vectors: *Specification*

```
impl RVec<T> {  
    // create a new refined vector of size 0  
    fn new() → RVec<T>[0];  
  
    // retrieve the size of the vector  
    fn len(&self[@n]) → usize[n];  
  
    // get/set element at index i statically checking bounds  
    fn get(&self[@n], i: usize{v: v < n}) → &T;  
    fn set(&mut self[@n], i: usize{v: v < n}, val: T) → &T;  
  
    // insert element at the end of the vector  
    fn push(&mut self[@n], val: T) ensures self: RVec<T>[n+1];  
}
```

21st Century Verification

Iterations, Collections, λ -Abstractions, ...

Iteration using while

```
fn dot_product(x:&Vec<f64>, y:&Vec<f64>)
    → f64
    requires x.len() == y.len(),
{
}
}
```

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n])
    → f64
{
}
}
```

Same index specifies same size precondition

Iteration using while

```
fn dot_product(x:&Vec<f64>, y:&Vec<f64>)
  → f64
  requires x.len() == y.len(),
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
    invariant
      i <= x.len(),
      x.len() == y.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n])
  → f64
{
  let mut sum = 0.0;
  let mut i = 0;
  while i < x.len()
  {
    sum += x[i] * y[i];
    i += 1;
  }
  sum
}
```

Invariant via *immutable type*

Iteration with **for** loop

```
fn dot_product(x:&Vec<f64>[@n], y:&Vec<f64>[n]) -> f64
{
    let mut sum = 0.0;
    for i in 0..x.len() {    // iterator guarantees
        sum += x[i] * y[i];  //    0 ≤ i < n
    }
    sum
}
```

Permits *range* iteration instead of old school while

21st Century Verification

Iterations, *Collections*, λ -Abstractions, Datatypes,...

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>)
    → Vec<f64>
requires
     $\forall i: \text{int} \mid 0 \leq i < \text{ws.len()} \implies \text{ws}[i].\text{len()} == \text{ins.len()}()$ 
{
}
```

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>)
    → Vec<f64>
{
}
}
```

Quantified contracts via *type composition*

Collections

```
fn forward(ws: &Vec<Vec<f64>>, ins: &Vec<f64>)
  → Vec<f64>
  requires
     $\forall |i: \text{int}| \ 0 \leq i < \text{ws.len()} \Rightarrow \text{ws}[i].\text{len()} = \text{ins.len()}$ 
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
    invariant
       $i \leq \text{ws.len()},$ 
       $\forall |j: \text{int}| \ 0 \leq j < \text{ws.len()} \Rightarrow \text{ws}[j].\text{len()} == \text{ins.len()}$ 
    {
      let weighted_input = dot_product(&ws[i], ins);
      res.push(sigmoid(weighted_input));
      i += 1;
    }
  res
}
```

```
fn forward(ws: &Vec<Vec<f64>[n]>, ins: &Vec<f64>[@n])
  → Vec<f64>
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < ws.len()
  {
    let weighted_input = dot_product(&ws[i], ins);
    res.push(sigmoid(weighted_input));
    i += 1;
  }
  res
}
```

Quantified invariants via *syntactic typing*

21st Century Verification

Iterations, Collections, *λ -Abstractions*, Datatypes, ...

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
    ∀ |i:int| 0 ≤ i < n ==> call_requires(f, i)
  ensures
    res.len() == n,
    ∀ |i:int| 0 ≤ i < n ==> call_ensures(f, i, res[i])
{
}
```

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{
}
```

Higher-order contracts via *parametric polymorphism*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>
  requires
     $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_requires}(f, i)$ 
  ensures
    res.len() == n,
     $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_ensures}(f, i, \text{res}[i])$ 
{
  let mut res = Vec::new();
  let mut i = 0;
  while i < n
    invariant
      i <= n, res.len() == i,
       $\forall |i: \text{int}| 0 \leq i < n \Rightarrow \text{call\_requires}(f, i)$ ,
       $\forall |j: \text{int}| 0 \leq j < i \Rightarrow \text{call\_ensures}(f, j, \text{res}[j])$ 
    {
      res.push(f(i));
      i += 1;
    }
  res
}
```

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{
  let mut res = Vec::new(); // res: Vec<A>[0]
  let mut i = 0;
  while i < n                // res: Vec<A>[i]

  {
    res.push(f(i));
    i += 1;
  }
  res                        // res: Vec<A>[n]
}
```

Quantifier-free Invariant via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]  
{ /* ... */ }
```

Polymorphic Instantiation via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{ /* ... */ }

// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize)
  → Vec<Vec<f64>[in_size]>[out_size]
{
  init(out_size, |_| { // A := Vec<f64>[in_size]
    init(in_size, |_| random()) // A := f64
  })
}
```

Polymorphic Instantiation via *refinement synthesis*

λ -Abstractions

```
fn init<A>(n: usize, mut f: impl FnMut(usize) → A) → Vec<A>[n]
{ /* ... */ }

// Create a 2-d out_size x in_size weight matrix
fn mk_weights(in_size: usize, out_size: usize)
  → Vec<Vec<f64>[in_size]>[out_size]
{
  init(out_size, |_| { // A := Vec<f64>[in_size]
    init(in_size, |_| random()) // A := f64
  })
}
```

Polymorphic Instantiation via *refinement synthesis*

21st Century Verification

Iterations, Collections, λ -Abstractions, *Datatypes*, ...

Datatypes

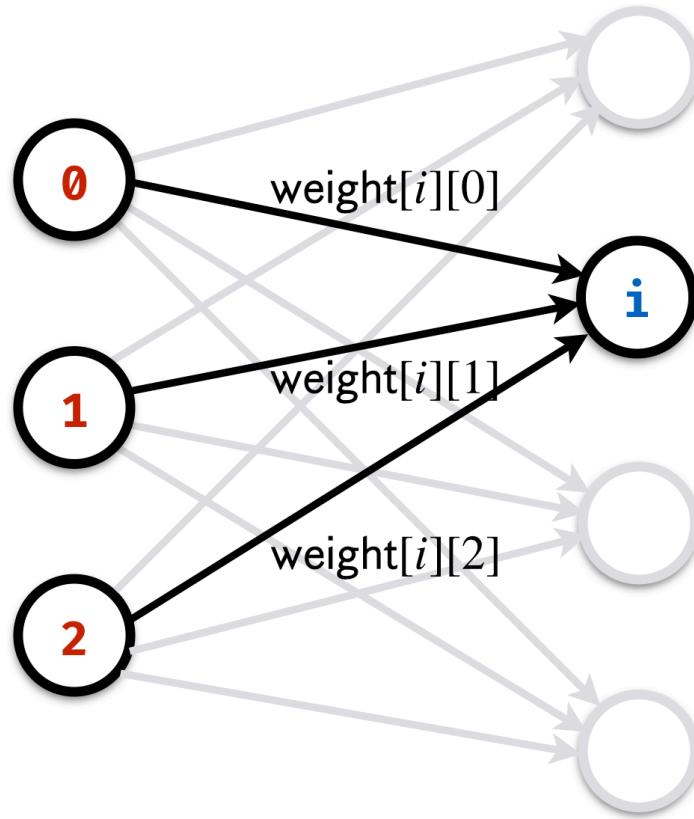
`struct`

“Product” types

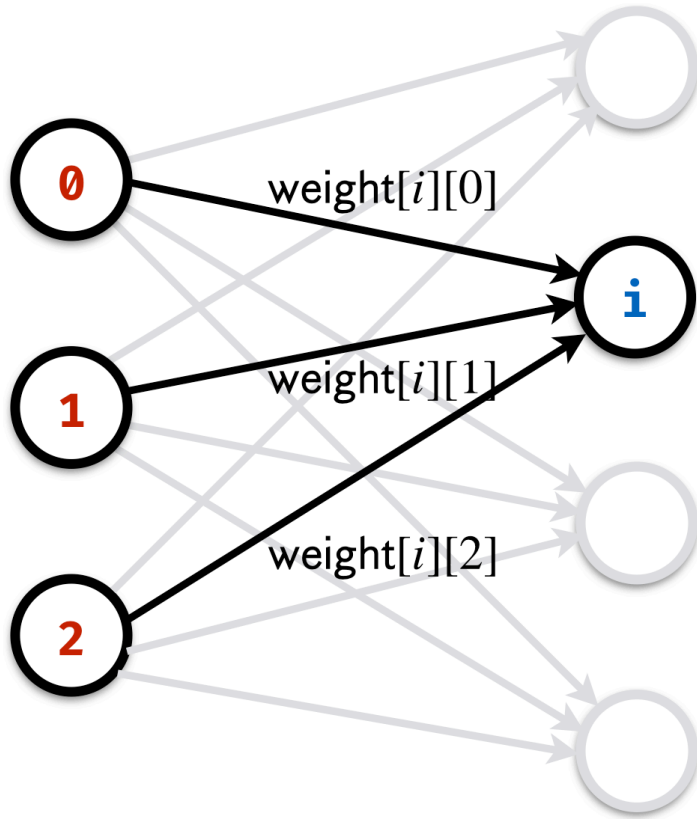
`enum`

“Sum” types

Example: *Neuron Layer*

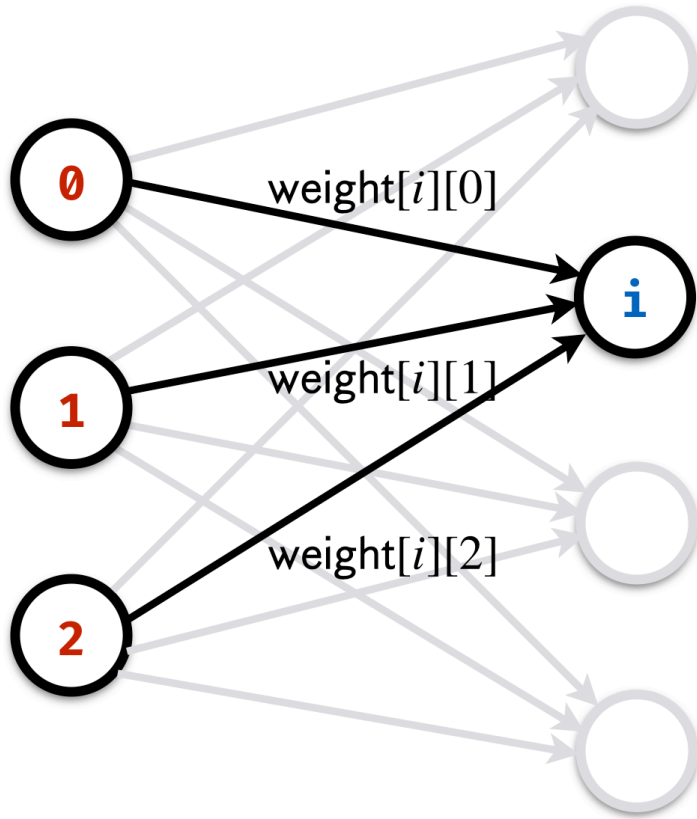


Neuron Layer: *Specification*



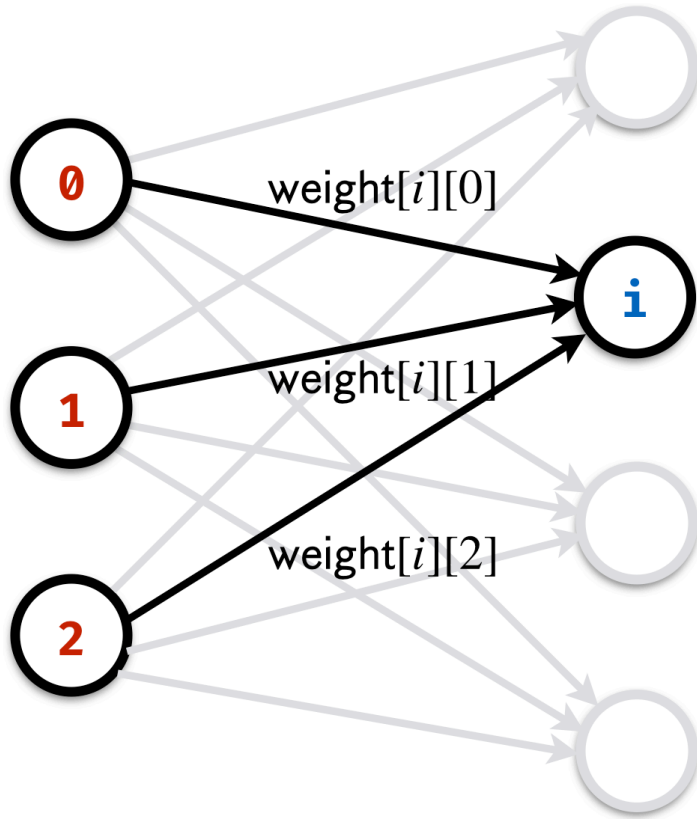
```
#[refined_by(i: int, o: int)]  
struct Layer {
```

Neuron Layer: *Specification*



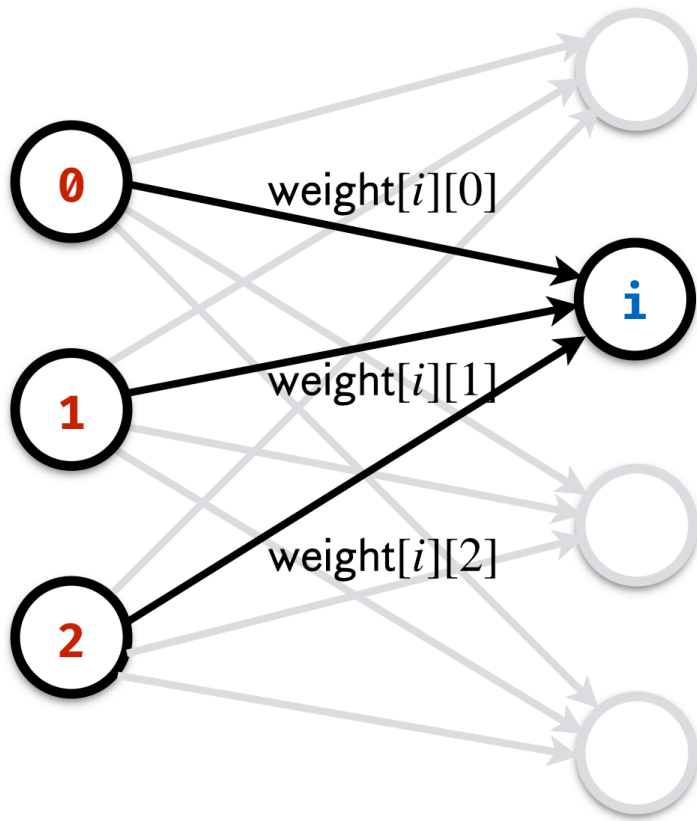
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
}
```

Neuron Layer: *Specification*



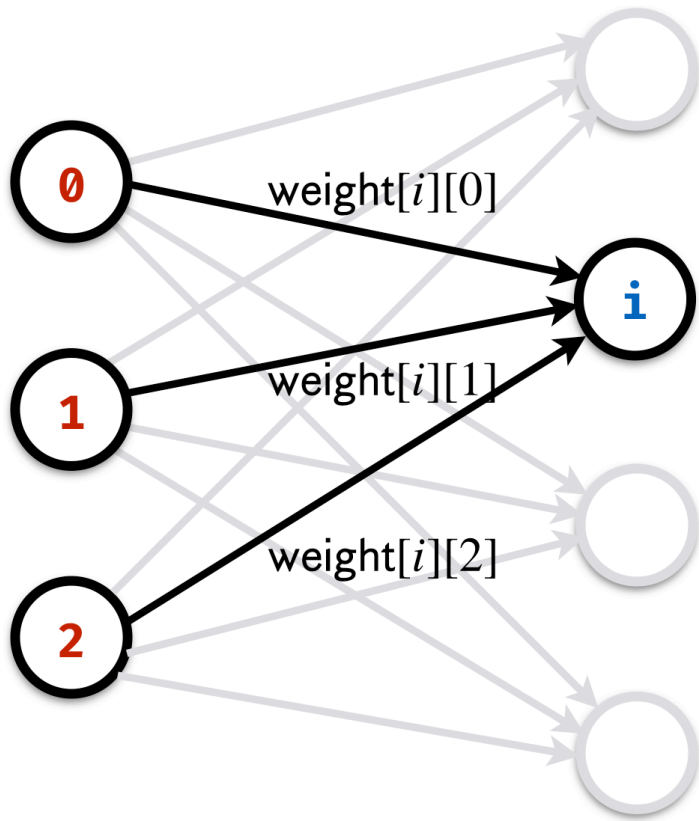
```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],
```

Neuron Layer: *Specification*



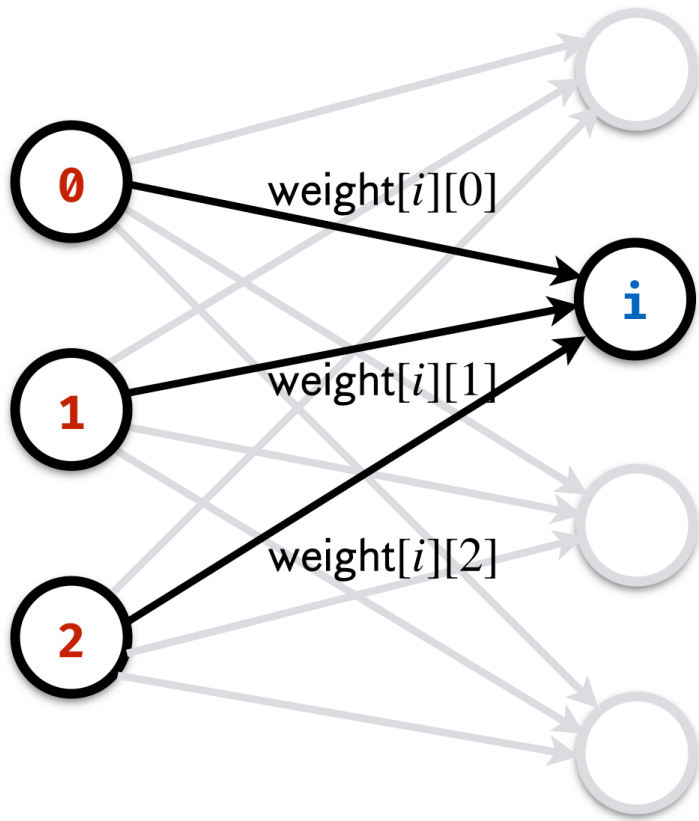
```
#[refined_by(i: int, o: int)]
struct Layer {
    num_inputs: usize[i],
    num_outputs: usize[o],
    weight: RVec<RVec<f64>[i]>[o],
    bias: RVec<f64>[o],
```

Neuron Layer: *Specification*



```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
    outputs: RVec<f64>[o],  
}
```

Neuron Layer: *Specification*

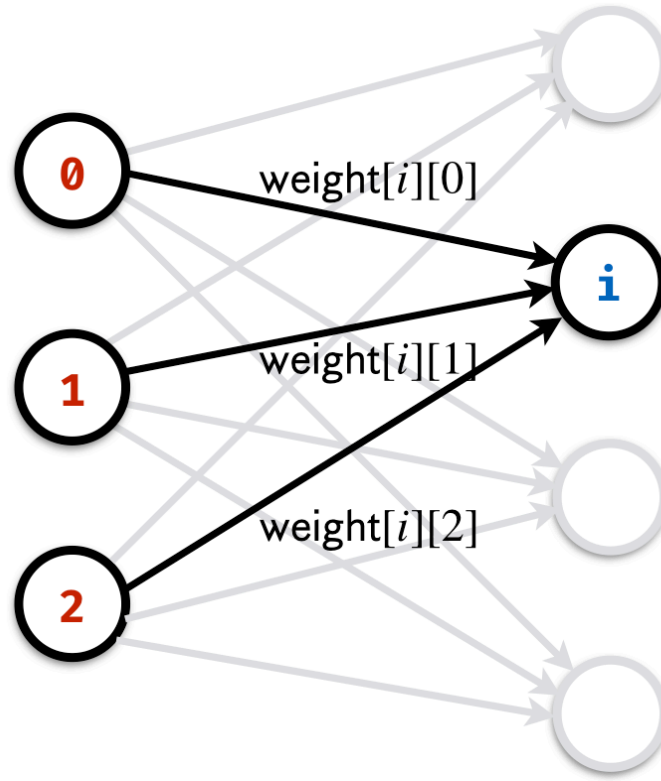


```
#[refined_by(i: int, o: int)]  
struct Layer {  
    num_inputs: usize[i],  
    num_outputs: usize[o],  
    weight: RVec<RVec<f64>[i]>[o],  
    bias: RVec<f64>[o],  
    outputs: RVec<f64>[o],  
}
```

Neuron Layer: *Verification*

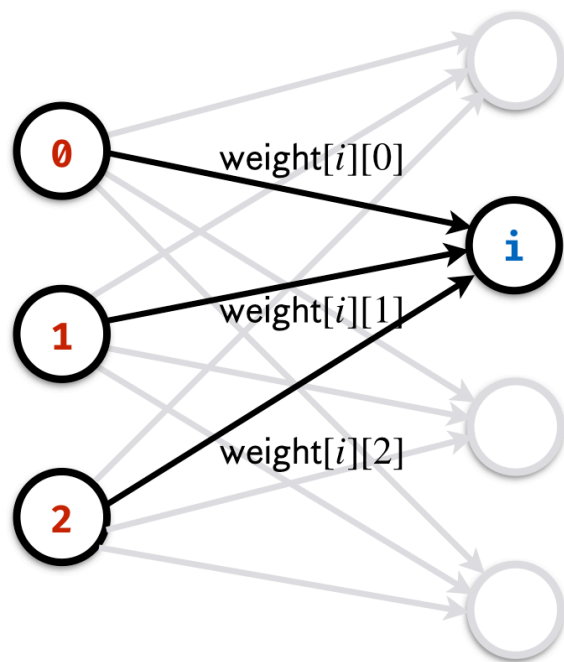
```
fn new(i: usize, o: usize) → Layer[i, o] {  
  let mut rng = rand::thread_rng();  
  Layer {  
    num_inputs: i,  
    num_outputs: o,  
    weight: init(o, |_| init(i, |_| rng.gen_range(-1.0..1.0))),  
    bias: init(o, |_| rng.gen_range(-1.0..1.0)),  
    outputs: init(o, |_| 0.0),  
  }  
}
```

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

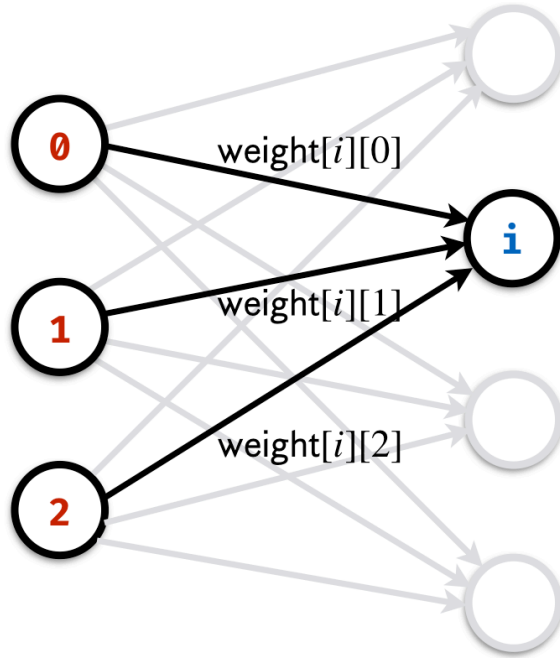
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

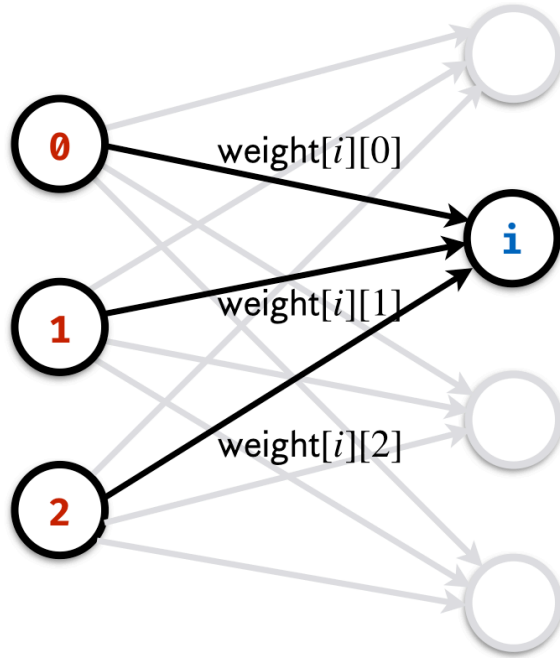
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

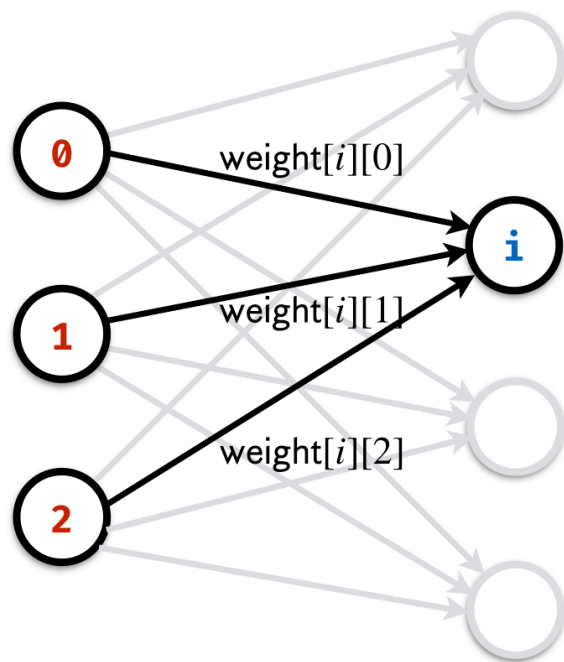
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

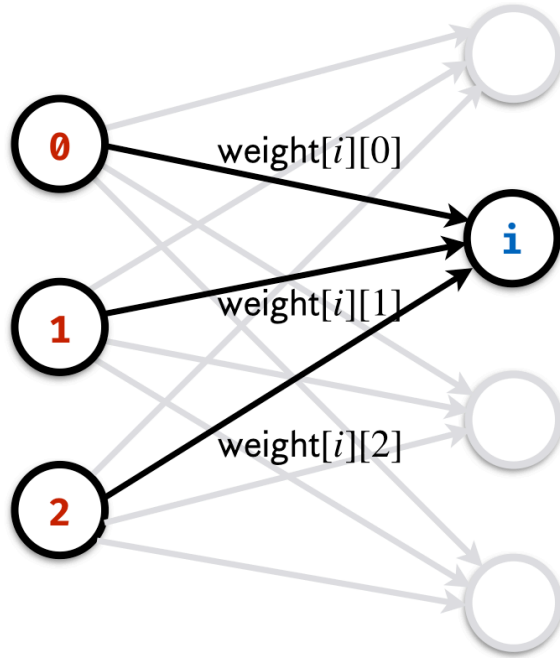
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

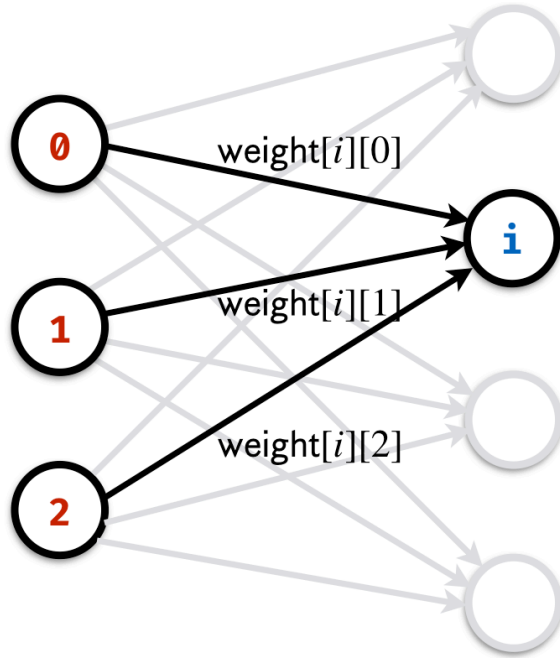
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

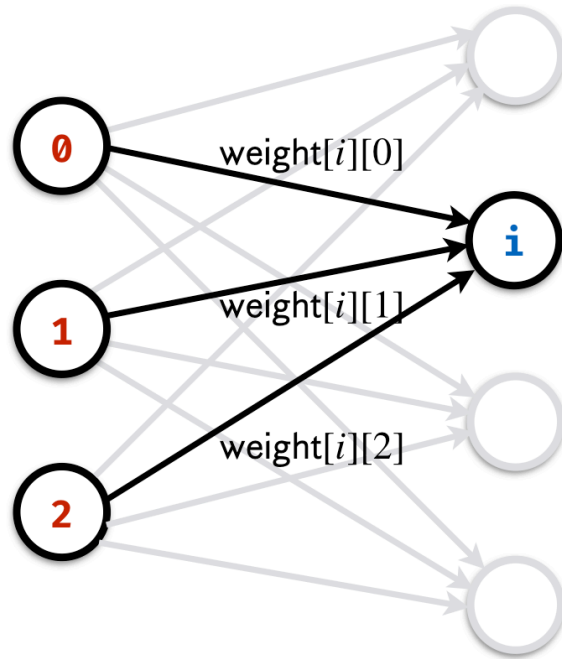
Neuron Layer: *Forward Propagation*



```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

Neuron Layer: *Forward Propagation*



$$\text{out}[i] \doteq \sigma(\text{in} \cdot \text{weight}[i] + \text{bias}[i])$$

```
fn forward(&mut self, input: &RVec<f64>) {  
    (0..self.num_outputs).for_each(|i| {  
        let wt = dot(&self.weight[i], input);  
        let sum = wt + self.bias[i];  
        self.outputs[i] = sigmoid(sum);  
    })  
}
```

Exercise: Can you *fix* the error?

Datatypes

struct

“Product” types

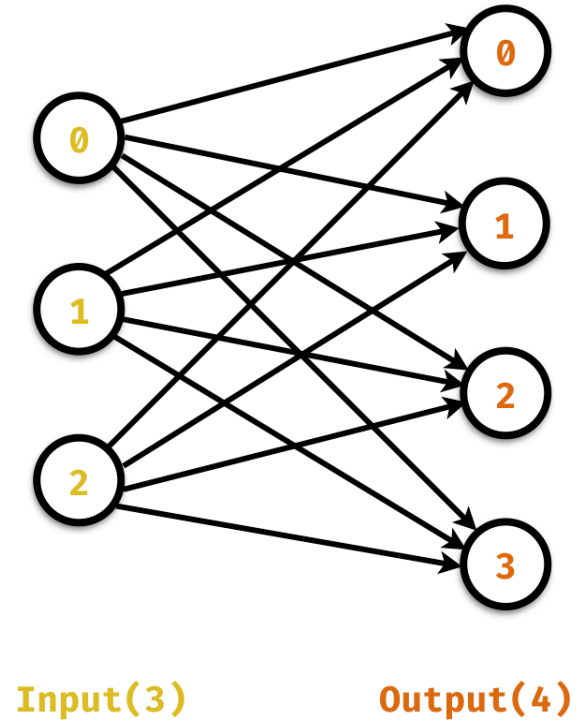
enum

“Sum” types

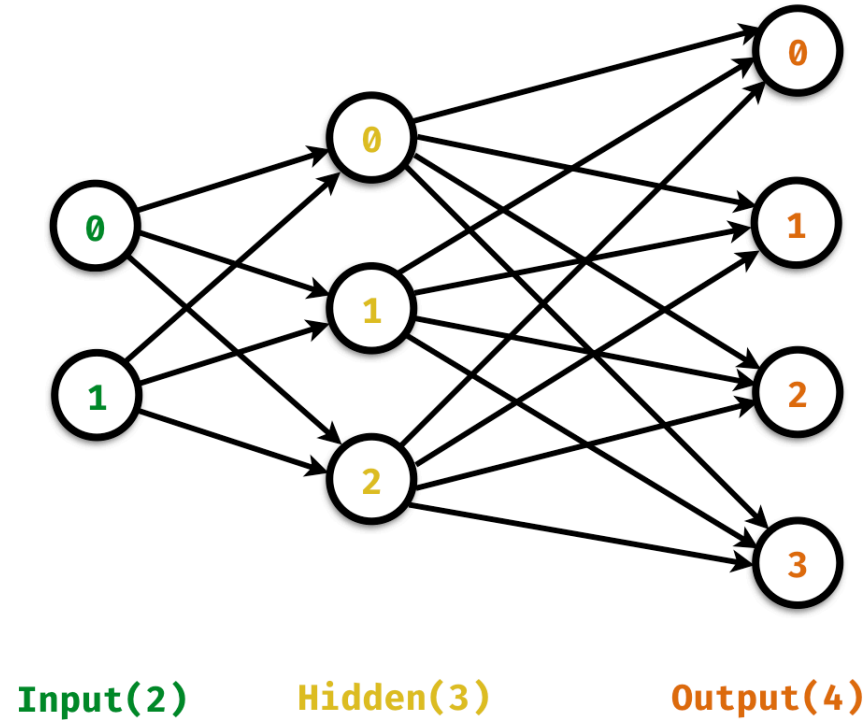
enum

Example: *Neural Network*

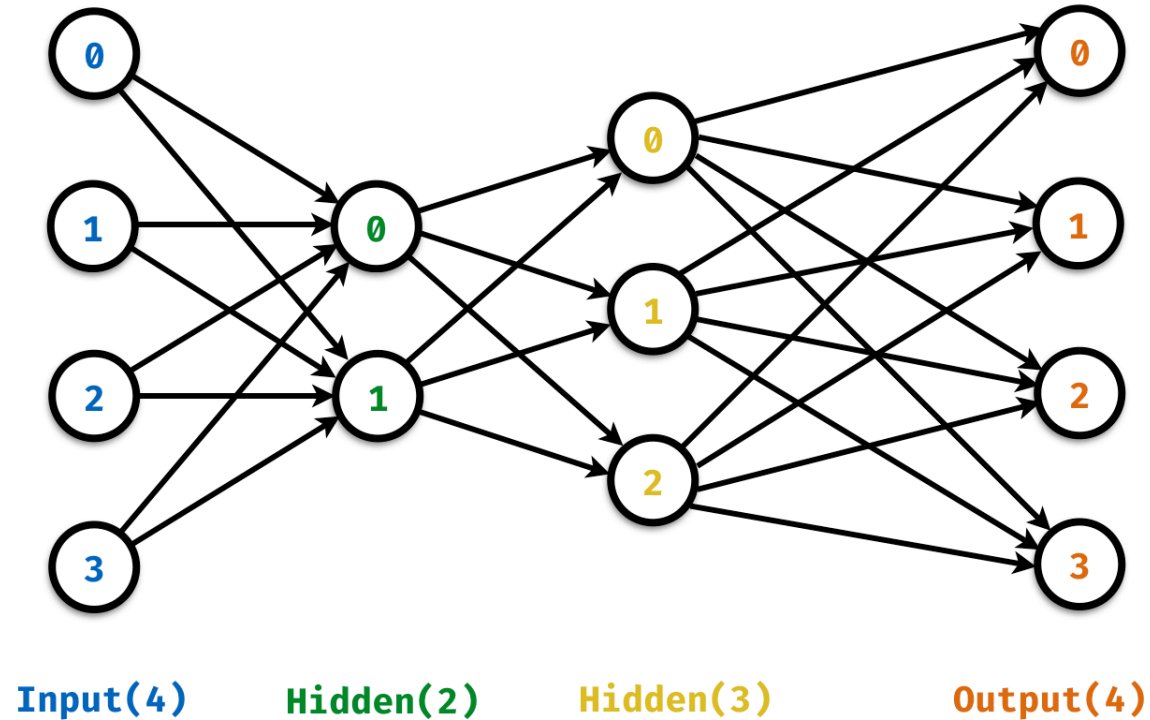
Neural Network has *Many* Layers



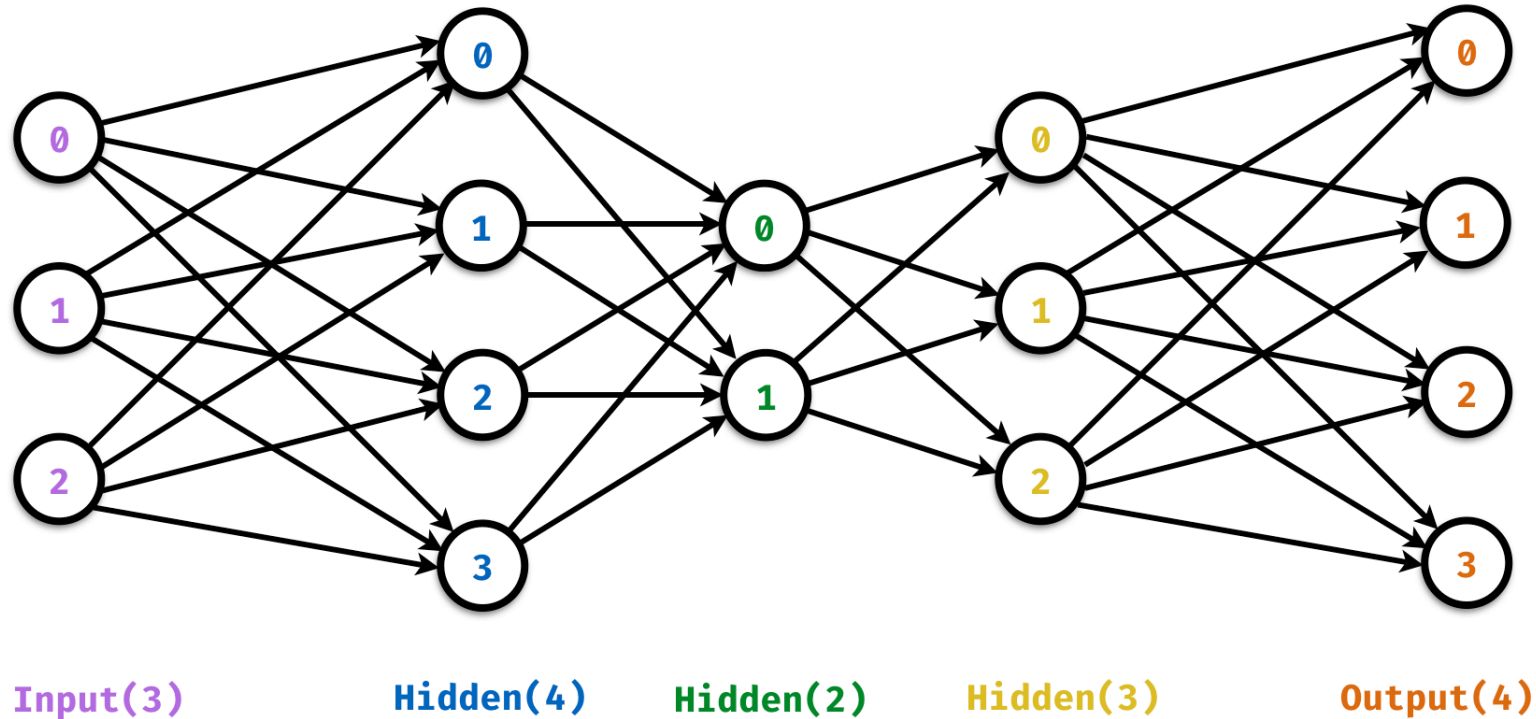
Neural Network has *Many* Layers



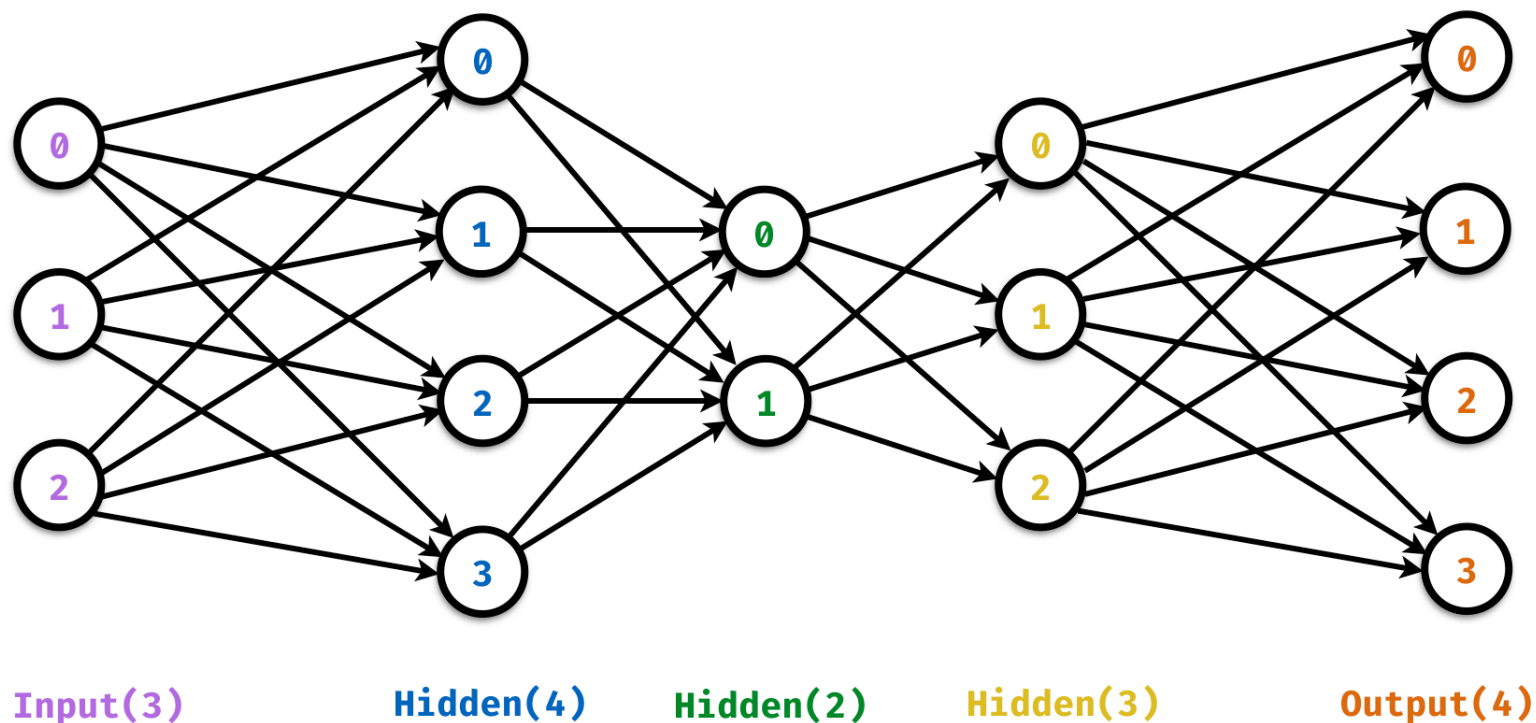
Neural Network has *Many* Layers



Neural Network has *Many* Layers



Neural Network has *Many* Layers



How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
enum Network {  
    Last(Layer),  
    Next(Layer, Box<Network>),  
}
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],
```

How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],  
    Next(Layer[@i, @n], Box<Network[n, @o]>) → Network[i, o],  
}
```

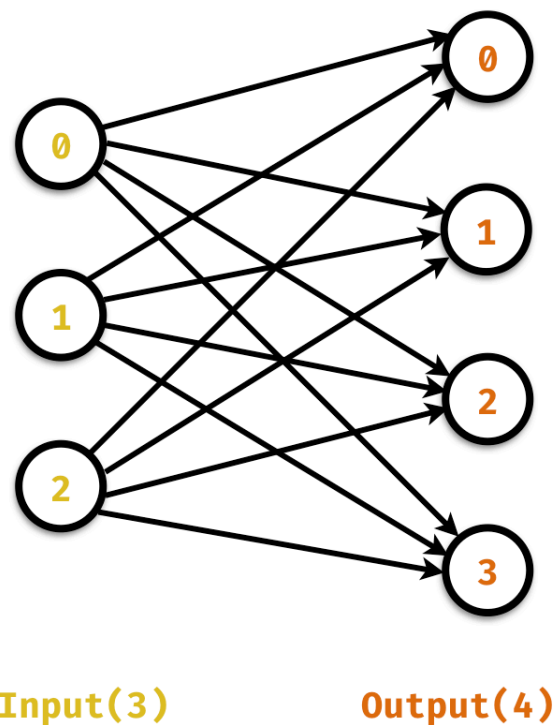
How to ensure layers *compose* correctly?

Neural Network: *Specification*

```
#[refined_by(i: int, o: int)]  
enum Network {  
    Last(Layer[@i, @o]) → Network[i, o],  
    Next(Layer[@i, @n], Box<Network[n, @o]>) → Network[i, o],  
}
```

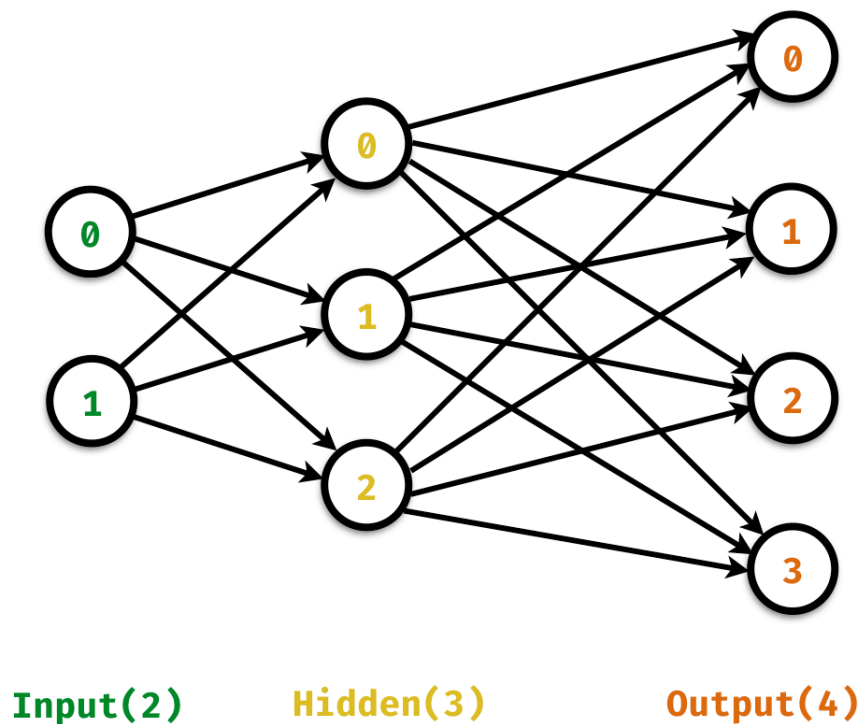
How to ensure layers *compose* correctly?

Refinements Ensure Correct *Composition*



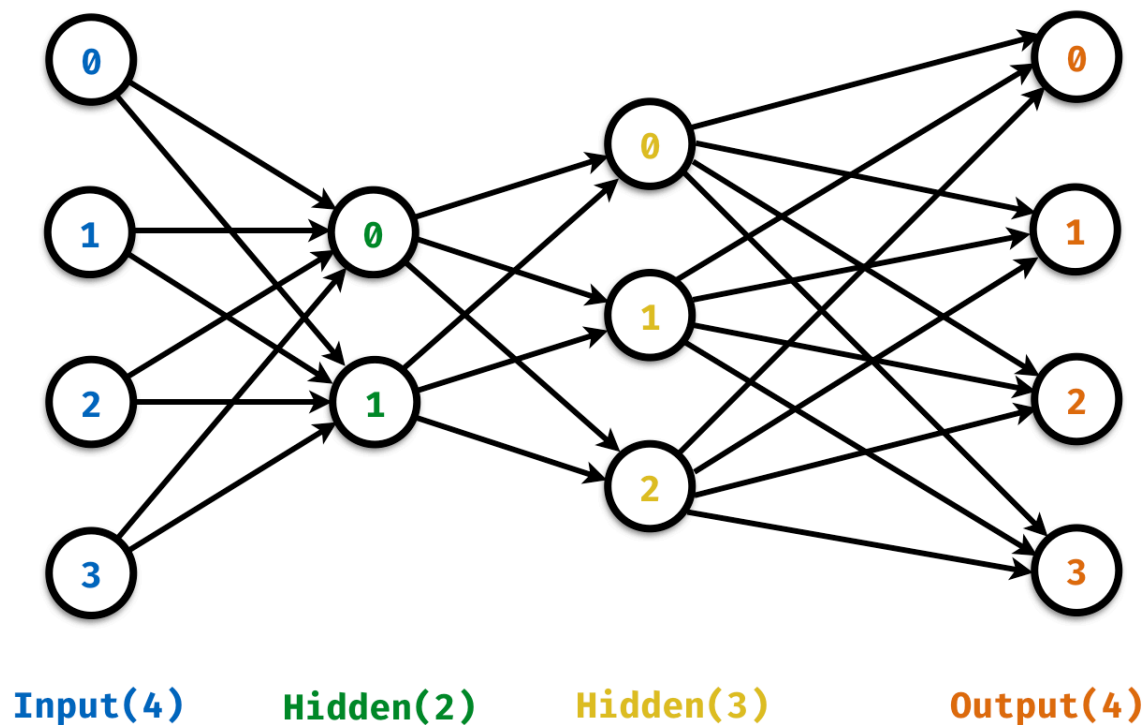
`Last(Layer[3,4])` $\longrightarrow$ `Network[3, 4]`

Refinements Ensure *Correct Composition*



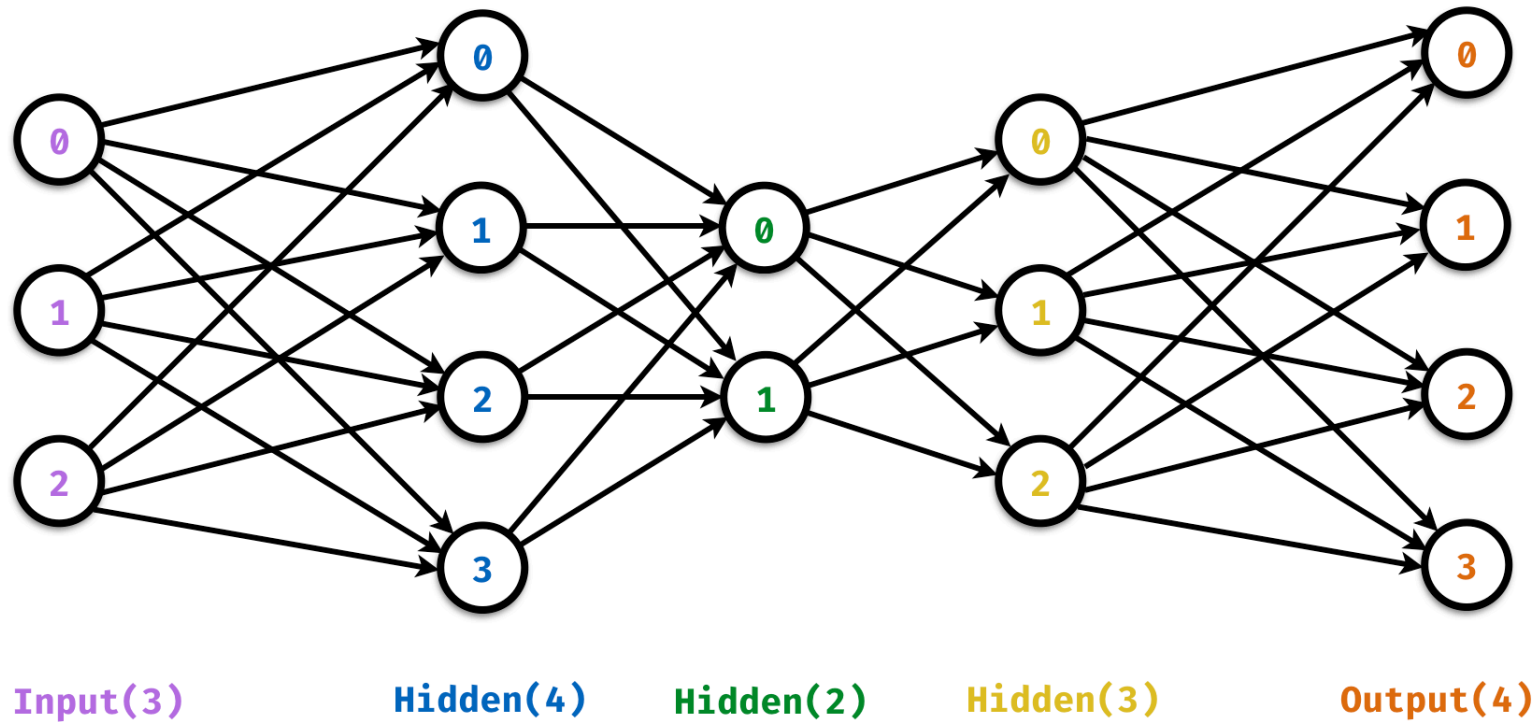
`Next(Layer[2,3], Network[3,4])  $\longrightarrow$  Network[2, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[4,2], Network[2,4])  $\longrightarrow$  Network[4, 4]`

Refinements Ensure Correct *Composition*



`Next(Layer[3,4], Network[4,4])  $\longrightarrow$  Network[4, 4]`

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)  
    → Network[input, output]
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn new(input: usize, hidden: &[usize], output: usize)
    → Network[input, output]
{
    if hidden_sizes.len() == 0 {
        Network::Last(Layer::new(input, output))
    } else {
        let n = hidden[0];
        let layer = Layer::new(input, n);
        let rest = Network::new(n, &hidden[1..], output);
        Network::Next(layer, Box::new(rest))
    }
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
    match self {  
        NeuralNetwork::Last(layer) ⇒ {  
            layer.forward(input);  
            layer.outputs.clone()  
        }  
        NeuralNetwork::Next(layer, next) ⇒ {  
            layer.forward(input);  
            next.forward(&layer.outputs)  
        }  
    }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
      next.forward(&layer.outputs)  
    }  
  }  
}
```

Neural Network: *Verification*

```
fn forward(&mut Network, input: &RVec<f64>) → RVec<f64> {  
  match self {  
    NeuralNetwork::Last(layer) ⇒ {  
      layer.forward(input);  
      layer.outputs.clone()  
    }  
    NeuralNetwork::Next(layer, next) ⇒ {  
      layer.forward(input);  
      next.forward(&layer.outputs)  
    }  
  }  
}
```

Exercise: *Fix* the specification for forward?

1. Problem

Program Logic *vs.* 21st century features

2. Solution

Refinements: *Weave* assertions into types

3. Verified Systems

Isolation in Tock OS Kernel

3. Verified Systems

Verified Systems

Tock

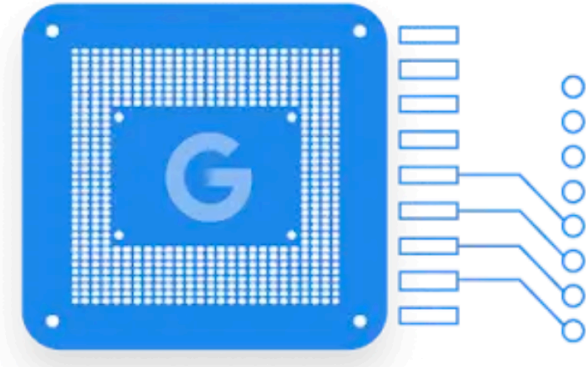
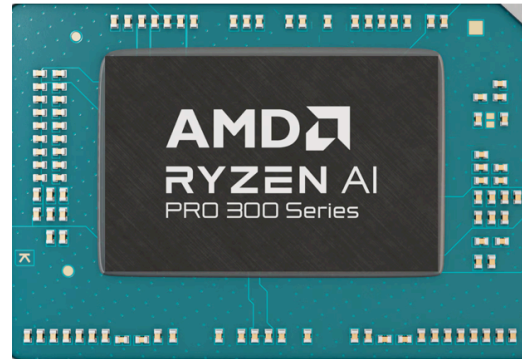
Verified Systems



An Embedded OS Kernel

Levy *et al.* SOSP 2017, Schuermann *et al.* SOSP 2025

Verified Systems



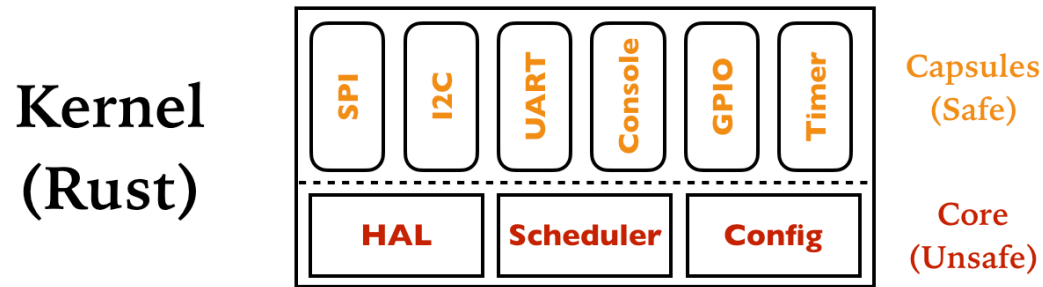
An Embedded OS Kernel

Firmware in Microsoft Pluton “root of trust”, Google ChromeBooks (GSC),...

Tock: *Architecture*

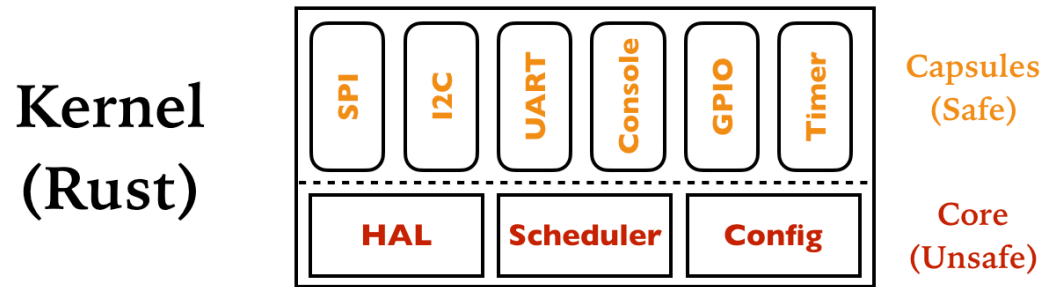
Tock: *Architecture*

Tock: Architecture



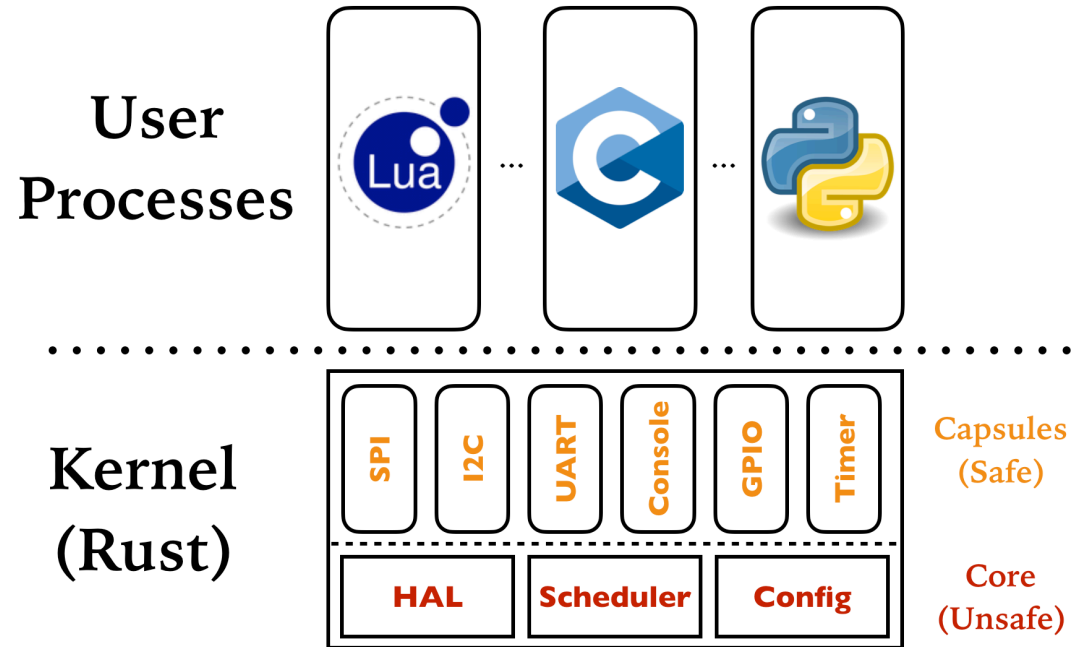
Capsules enjoy Rust's safety guarantees

Tock: Architecture



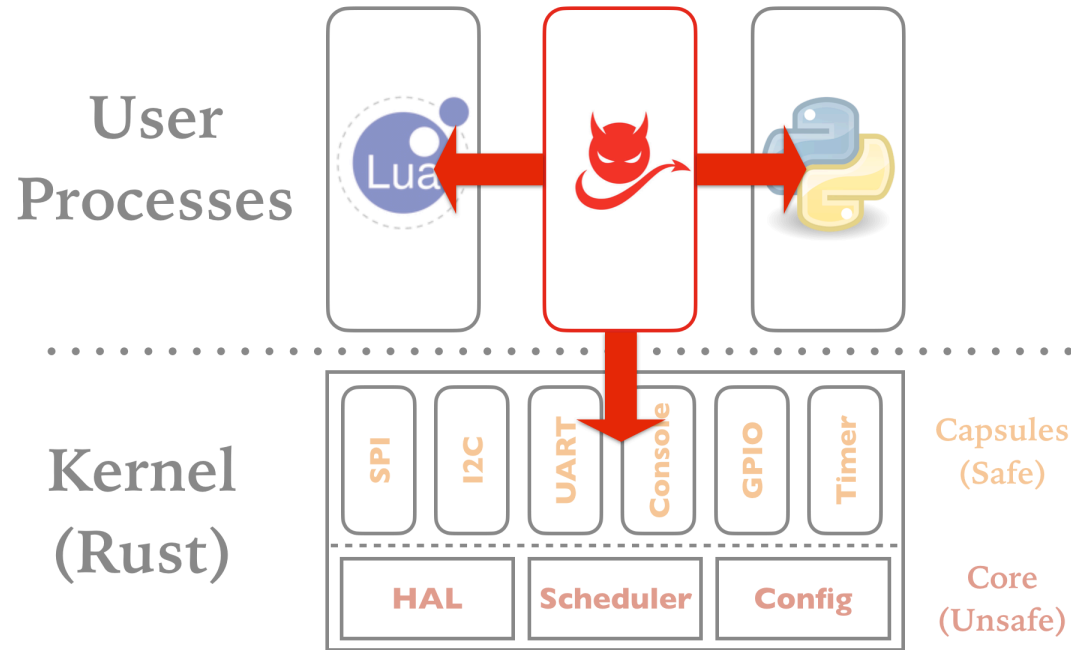
Core uses unsafe to manage hardware

Tock: Architecture



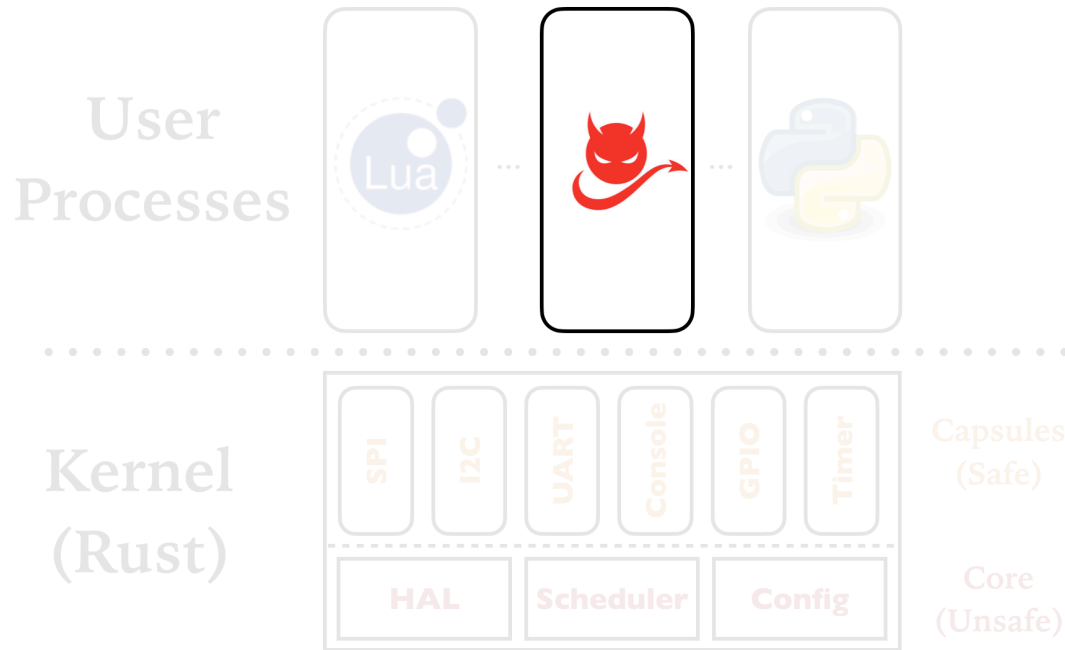
User processes can run *arbitrary code*

Tock: Architecture



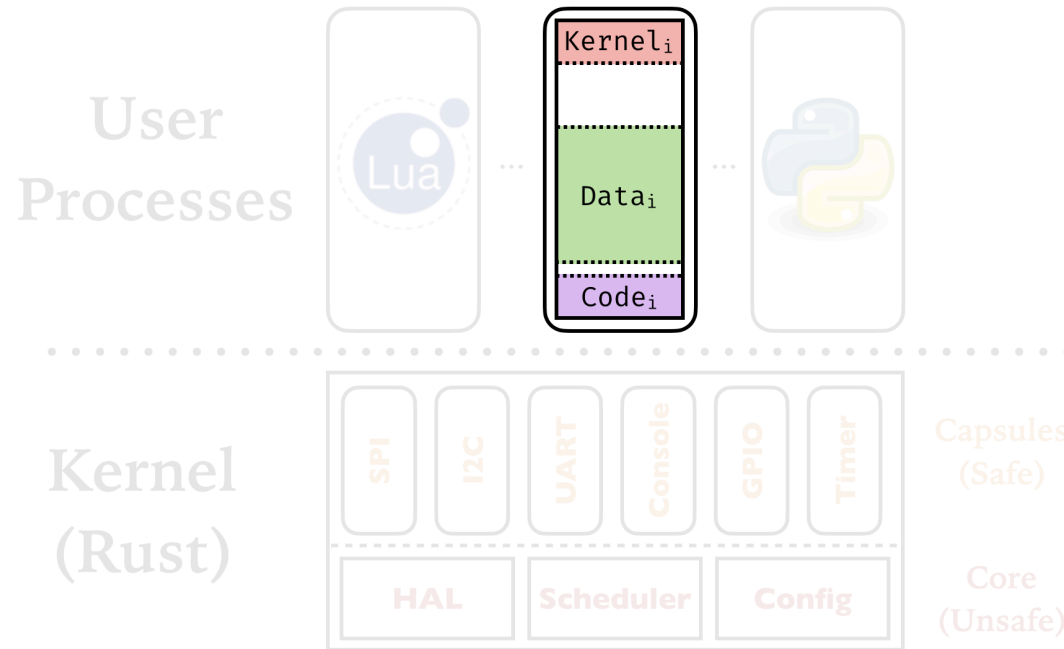
User processes run *arbitrary code* with vulnerabilities

Tock: Security by *Isolation*



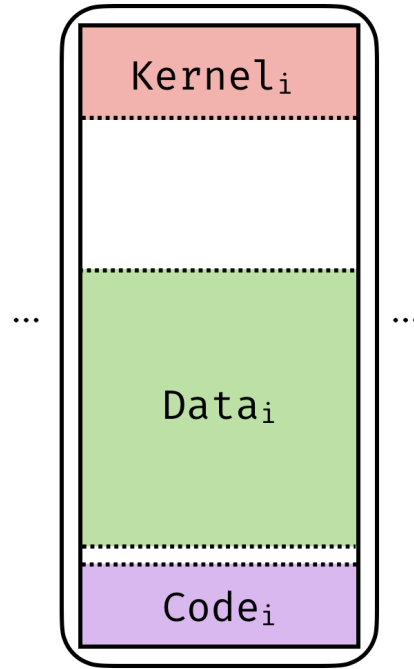
Isolate processes from kernel and each other

Tock: Security by *Isolation*



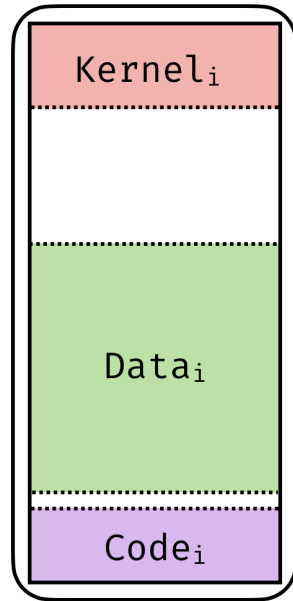
Process Memory: `Kernel` + `Data` + `Code`

Tock: Security by *Isolation*



Process Memory: $\text{Kernel} + \text{Data} + \text{Code}$

Tock: Security by *Isolation*



Ensure that **Proc_i**

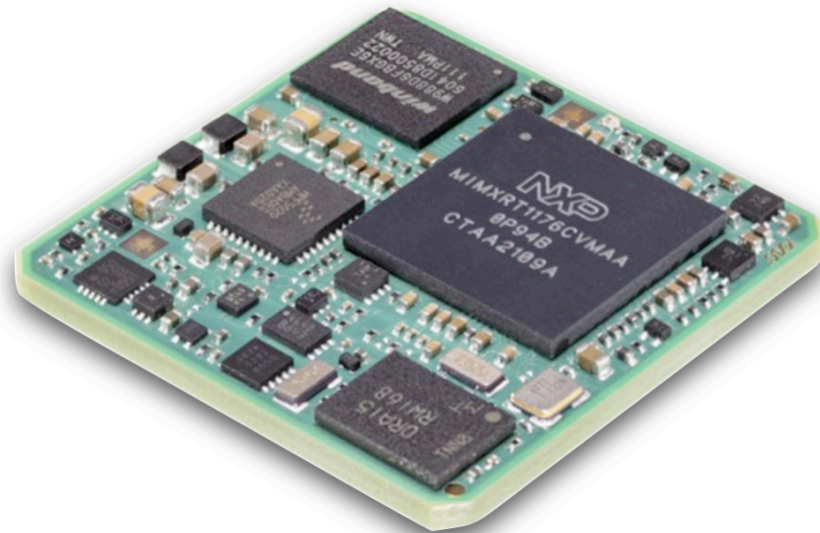
Cannot access Kernel_i

Can only Rd+Wr in Data_i

Can only Rd+Ex in Code_i

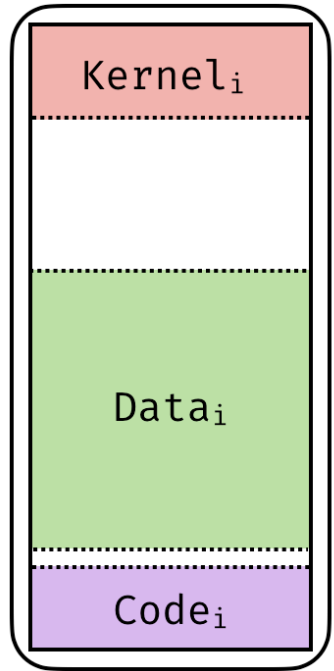
Process Memory: $\text{Kernel} + \text{Data} + \text{Code}$

Isolation via Memory Protection Units

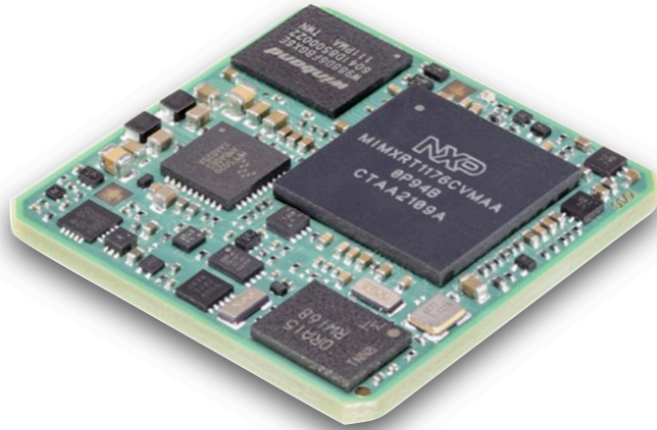
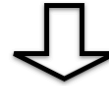


MPU controls address (regions) allowed to Rd, Wr, Ex

Isolation via Memory Protection Units



Configure MPU at switch to **Proc_i**



To *only* allow Rd+Wr in Data_i and Rd+Ex in Code_i

Isolation via Memory Protection Units

What could *possibly* go wrong?

Isolation via Memory Protection Units

What could *possibly* go wrong?

rv32i/{pmp,epmp}: invalidate unused regions on config switch #3541

 Merged bradjc merged 1 commit into [tock:master](#) from [lschuermann:otdev/pmp-invalidate-fix](#) on Jul 18, 2023

arch: cortex: fix MPU rounding calculation #2435

 Merged bors merged 2 commits into [master](#) from [cortex-mpu-rounding-calculation](#) on Feb 24, 2021

kernel: process: reconfigure mpu on restart #2146

 Merged bors merged 2 commits into [master](#) from [kernel-process-restart-mpu](#) on Oct 23, 2020

Potential sharing of kernel memory #1141

 Closed  #1144

Cortex-M MPU: [allocate_app_memory_region](#) allows access to kernel grant memory #4366

 Closed  #4384

riscv: pmp: disallow access above app brk #2173

 Merged bradjc merged 1 commit into [master](#) from [pmp-exclude-grant-region](#) on Oct 23, 2020

arch/rv32i: pmp/ePMP: Fixup PMP comparision #2947

 Merged bors merged 2 commits into [tock:master](#) from [alistair23:alistair/pmp-fixup](#) on Jan 28, 2022

rv32: pmp: Fix bounds #2076

 Merged bors merged 1 commit into [master](#) from [pmp-address-fix](#) on Aug 19, 2020

What could *possibly* go wrong?

Buggy *MPU configuration*

What could *possibly* go wrong?

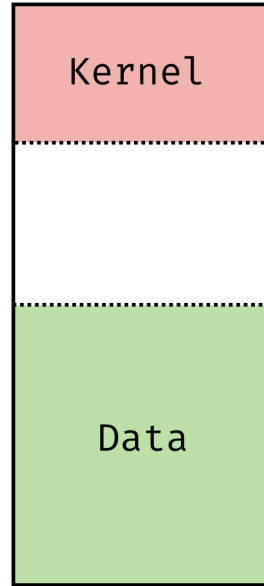
Buggy *MPU configuration*

Buggy *interrupts & context switching*

Buggy *MPU Configuration*

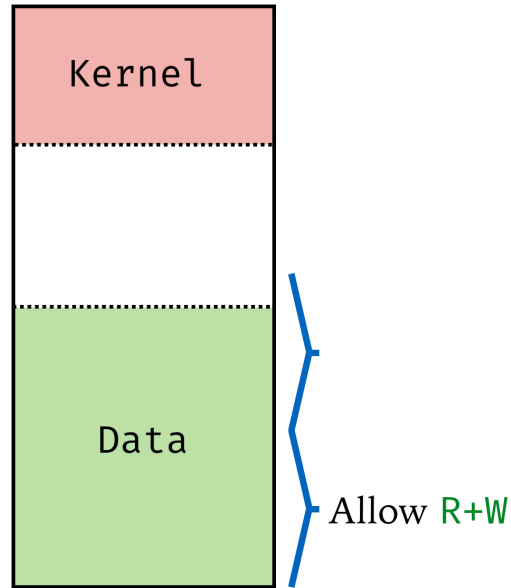
MPUs permit access control over *fixed size regions*

Buggy *MPU Configuration*



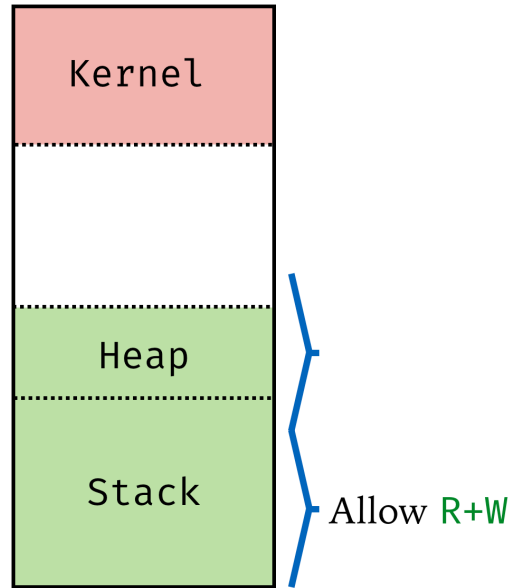
MPUs permit access control over *fixed size regions*

Buggy MPU Configuration



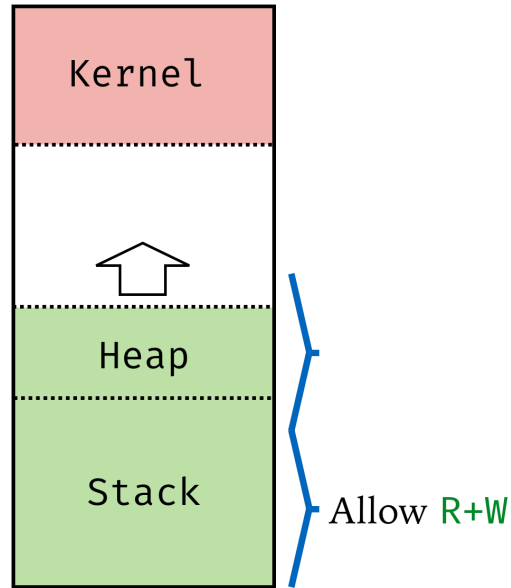
MPUs permit access control over *fixed size regions*

Buggy *MPU Configuration*



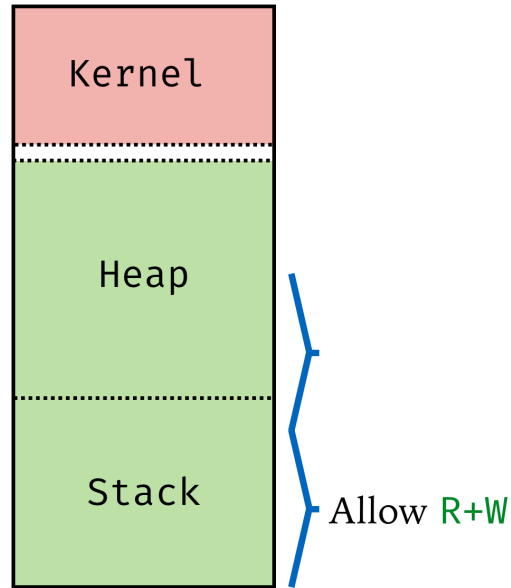
Process Accesses Heap + Stack

Buggy *MPU Configuration*



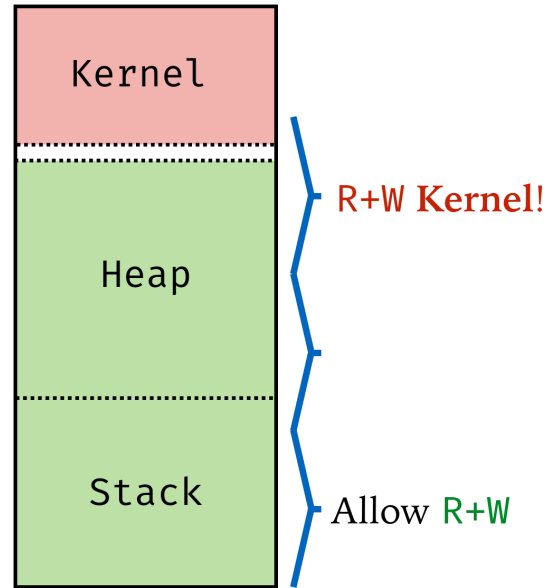
Heap can *grow* via kernel syscall

Buggy *MPU Configuration*



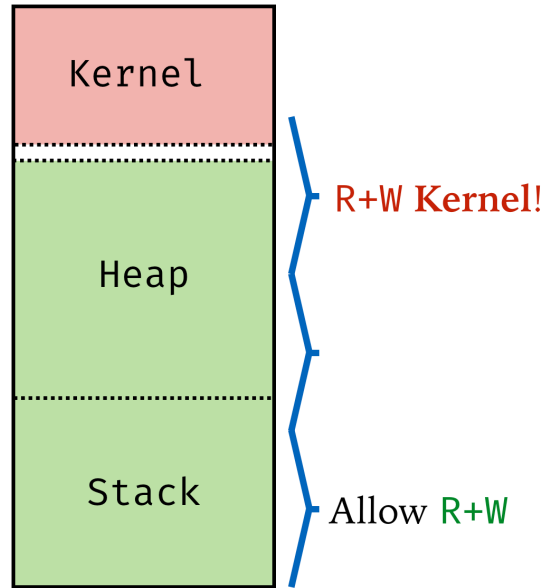
Heap can *grow* via kernel syscall

Buggy MPU Configuration



Covering *regions* lets process R+W kernel data!

Buggy *MPU Configuration*



Challenge: Ensure *gap* to avoid overlap!

Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Refine kernel's type for *process memory*

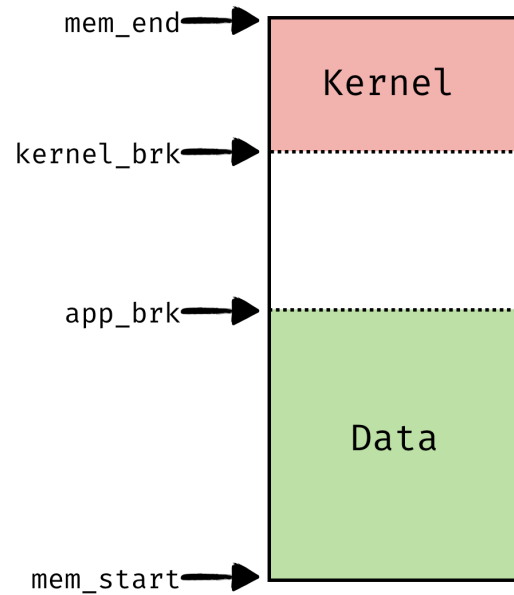
Verified MPU Configuration

Challenge: Ensure *gap* to avoid overlap!

Refine kernel's type for *process memory*

Refine MPU's type for *region configuration*

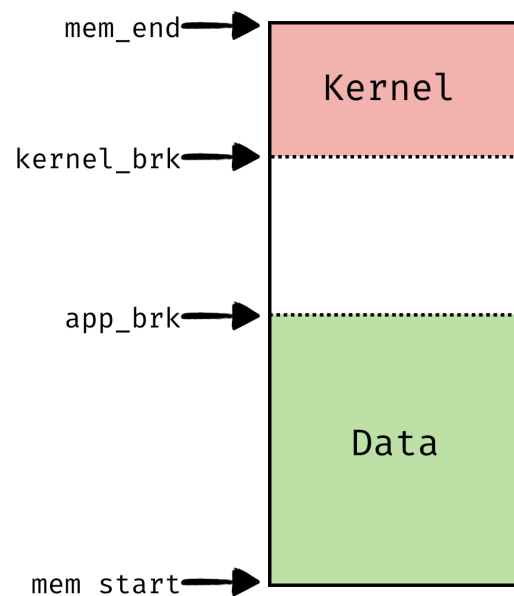
Verified MPU Configuration



Refine kernel's type for *process memory*

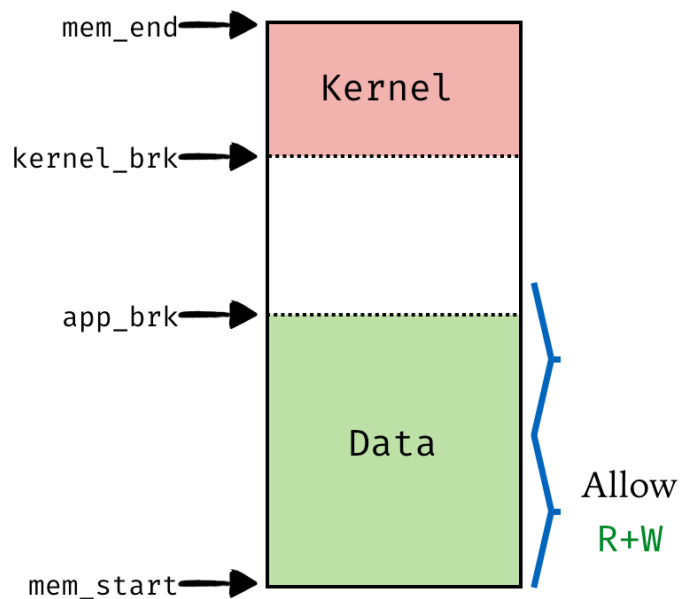
Verified MPU Configuration

```
struct ProcessMemory
{
    mem_end: Ptr[v: mem_start < v],
    kernel_brk: Ptr[v: app_brk < v],
    app_brk: Ptr[v: mem_start <= v],
    mem_start: Ptr,
}
```



Refine kernel's type for *process memory*

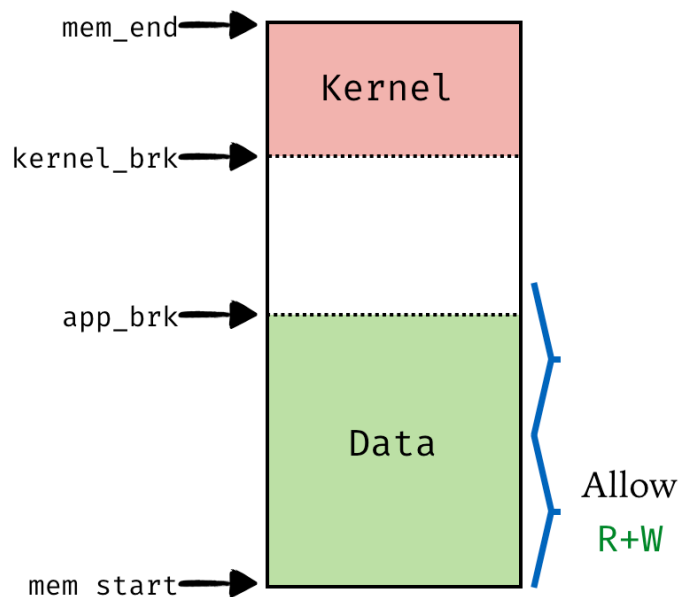
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}
```

Refine MPU's type for *region configuration*

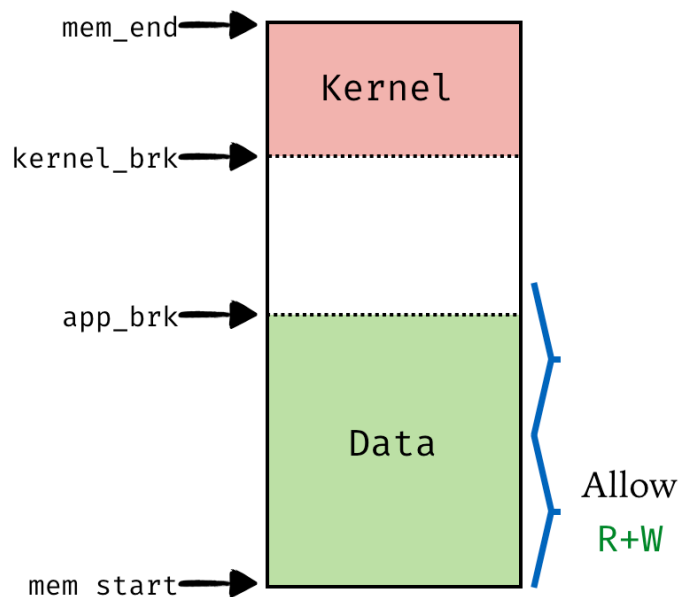
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
    can_access_ram(breaks, regions) &&  
    cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

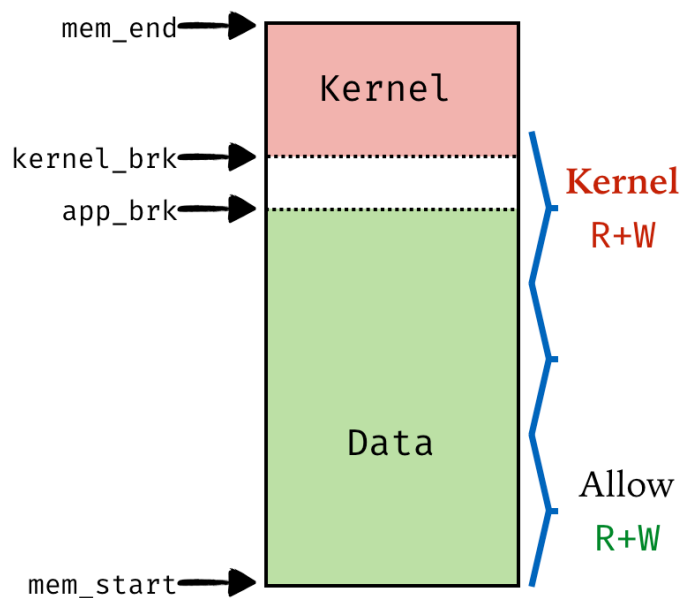
Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
    brks: Breaks,  
    regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
    can_access_ram(breaks, regions) &&  
    cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

Verified MPU Configuration



```
struct RegionConfig<R: Region>,  
{  
  brks: Breaks,  
  regions: Array<R>{v: ok_regions(brks, v)}  
}  
  
fn ok_regions(brks, regions) → Bool {  
  can_access_ram(breaks, regions) &&  
  cannot_access_other(breaks, regions)  
}
```

Refine MPU's type for *region configuration*

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified Interrupts & Context switching

Interrupts can break isolation in various ways

Verified Interrupts & Context switching

Interrupts can break isolation in various ways

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

Closed#4259



vrindisbacher opened on Nov 21, 2024 · edited by vrindisbacher

Edits Contributor ...

I'm opening this issue following the discussion in [#4242](#).

I believe that the following ISRs need to set the special `CONTROL` register to `0` before exception return, similar to [the systick_handler](#) on [line 193](#)

- [Generic ISR](#)
 - I think that this is canceled out by [ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245](#), since we return to the process rather than the kernel when coming from a process.
- [SVC Handler](#)
 - My hunch here is that we should be setting thread mode to privileged execution in the case we are coming from a process and need to return to the kernel to handle a syscall. I think setting the `CONTROL` register to `0` [here](#) should do the trick. Similar to how the `v7m svc handler` does [here](#).
 - It might also be a good idea to ensure that we set thread mode to unprivileged execution when we go from kernel to process. It seems like the `v7m svc handler` does this [here](#).



 alevy added bug on Nov 27, 2024

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FLUXARM

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FluxARM

ARM

```
systick_handler:  
  
    mov r0, #0  
    msr CONTROL, r0  
    isb  
    ldr lr, =0xFFFFFFF9  
    bx lr
```



FluxARM

```
fn systick_handler(self: &mut Arm7[@old])  
    -> BV32[0xFFFF_FFF9]  
    ensures { ... }  
{  
    self.movw_imm(GPR::R0, 0);  
    self.msr(Control, GPR::R0);  
    self.isb();  
    self.ldr(lr(), 0xFFFF_FFF9);  
    self.bx(lr)  
}
```

Verified Interrupts & Context switching

Lift ARM assembly to *Refined* Rust methods FluxARM

ARM

```
systick_handler:  
  
    mov r0, #0  
    msr CONTROL, r0  
    isb  
    ldr lr, =0xFFFFFFF9  
    bx lr
```



FluxARM

```
fn systick_handler(self: &mut Arm7[@old])  
    -> BV32[0xFFFF_FFF9]  
    ensures { ... }  
{  
    self.movw_imm(GPR::R0, 0);  
    self.msr(Control, GPR::R0);  
    self.isb();  
    self.ldr(lr(), 0xFFFF_FFF9);  
    self.bx(lr)  
}
```

Verify handlers save & restore kernel/MPU configuration

Verified *Isolation* in Tock [SOSP'25]

Verified *MPU configuration*

Verified *interrupts & context switching*

Verified *Isolation* in Tock [\[SOSP'25\]](#)

Component	Source	Fns(Trusted)	Specs(Trusted)
Kernel	11,974	550 (19)	355 (21)
ARM MPU	1,174	139 (14)	500 (17)
FLUX-STD	812	130 (48)	157 (60)
FLUXARM	3,413	95 (5)	1,908 (7)
Total	17,373	914 (86)	2,920 (105)

Specs = 6.5% lines of SLOC

Verified *Isolation* in Tock [SOSP'25]

Defensive programming in MPU driver API #4104

✓ Closed

🔗 #4135

ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245

✓ Closed

🔗 #4259

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

✓ Closed

🔗 #4259

Cortex-M MPU: `allocate_app_memory_region` allows access to kernel grant memory #4366

✓ Closed

🔗 #4384

App crashing allows potential sharing of kernel memory #4371

🔗 Open

Bug

Verification helped find & fix *six vulnerabilities* ...

Verified *Isolation* in Tock [SOSP'25]

Defensive programming in MPU driver API #4104

✓ Closed

🔗 #4135

ARMv6m: Register clobbering in Generic ISR causes exception exit to return to process rather than Kernel #4245

✓ Closed

🔗 #4259

ARMv6m: Thread mode not set to privileged execution in certain ISRs #4246

✓ Closed

🔗 #4259

Cortex-M MPU: `allocate_app_memory_region` allows access to kernel grant memory #4366

✓ Closed

🔗 #4384

App crashing allows potential sharing of kernel memory #4371

🔗 Open

Bug

... and a *clearer* design with a *faster* kernel

Program Verification for the 21st Century

1. Problem

Program Logic *vs.* 21st century features

2. Solution

Refinements: Weave **assertions** into **types**

3. Verified Systems

Isolation in Tock OS Kernel



<https://flux-rs.github.io>